Filo Yönetimi Uygulaması
V1.0

17.12.2025

İçindekiler

1. Dokümanın Amacı
2. Sistem Genel Mimarisi (Özet)
   2.1 Uygulamaya Genel Bakış
   2.2 Araç İzleme Modülü (Fleet Monitoring)
   2.3 Rota Optimizasyonu Modülü (Route Optimization)
      2.3.1 Müşteri Ekleme
      2.3.2 Algoritma Parametre Giriş ve Algoritma Seçim Ekranı
      2.3.3 Send to Route (EV, HIL, SUMO)
      2.3.4 Admin Yönetim Paneli
3. Hesaplar ve Yetkilendirme
   3.1 MongoDB Atlas
   3.2 Jira / Atlassian
4. Ağ ve Sunucu Bilgileri
   4.1 Statik IP
   4.2 Lokal Ağ (ESOGÜ Superbox)
5. Kullanılan Portlar
6. Firewall (Windows) Port Açma Örneği
7. Jetson Nano - FIWARE IP Güncelleme Adımları
8. Kod ve Yedekler
9. GitLab CI/CD Süreçleri Hak.
10. Fiware Ip Configuration
11. Uygulama Kurulum ve Servis Başlatma Adımları
   11.1 Veritabanı ve Gerekli Yazılımlar
   11.2 Backend Servisi Kurulumu
   11.3 Frontend Servisi Kurulumu ve Çalıştırılması
   11.4 A* Route Service Kurulumu
   11.5 SUMO Simülasyon Servisi
   11.6 Machine Prediction (Tahmin Servisi)
   11.7 Rota Aktarımı ve Fleet Monitoring Entegrasyonu
   11.8 Socket Server (8035) - Docker Kurulumu
   11.9 Algoritma Servisleri
   11.10 FIWARE Servisleri
   11.11 FIWARE Components Olası Arıza Durumu Hakkında
12. Kod Tabanı Genel Yapısı

12.1 Proje Mimarisi Genel Bakış
12.2 Veri Akış Özeti
12.4 Dosya Referansları
SİSTEM BAŞLATMA (start_all.bat)
BACKEND KLASÖRÜ
MODELS (Veritabanı Modelleri)
MIDDLEWARE
UTILS
musoshi_mock/ (Test Verileri)
FRONTEND KLASÖRÜ
src/ (Kaynak Kod)
src/pages/ (Sayfalar)
src/components/ (Bileşenler)
src/components/FleetMonitoring/
src/components/RouteOptimization/
src/contexts/ (React Context)
src/services/ (Servis Modülleri)
src/utils/ (Yardımcı Fonksiyonlar)
src/data/ (Statik Veri)
public/ (Statik Dosyalar)
MACHINE KLASÖRÜ (ML Servisi)
Client Dosyaları (Federated Learning)
app.py API Endpoint'leri
ROUTE-SERVICE KLASÖRÜ (Rota Servisi)
cache/ (Önbellek)
API Endpoint'leri
SUMO KLASÖRÜ (Simülasyon)
main.py API Endpoint'leri
algorithms/ (Algoritmalar)
ESOGU_SUMO/ (SUMO Dosyaları)
utils/ (Yardımcı Araçlar)
DEPLOY KLASÖRÜ (Kubernetes)
monitoring/ (İzleme Konfigürasyonları)
Alert Dosyaları
Exporter Dosyaları

PORT ÖZETİ .....
VERİTABANI BAĞLANTILARI .....
13. Backend Katmanı Detaylı İnceleme .....
13.1 Backend Klasör Yapısı .....
13.2 server.js (Ana Uygulama Giriş Noktası) .....
13.2.1 Modül İçe Aktarma ve Yapılandırma (Satır 1-50) .....
13.2.2 Kubernetes Bootstrap Entegrasyonu (Satır 32-60) .....
13.2.3 Otomatik Port-Forward Mekanizması (Satır 62-120) .....
13.2.4 CORS ve Middleware Yapılandırması (Satır 140-200) .....
13.2.5 Veritabanı Bağlantıları (Satır 250-290) .....
13.2.6 Socket ve FIWARE Entegrasyonu (Satır 140) .....
13.2.7 Route Tanımları ve API Endpoint'leri (Satır 300-4000+) .....
Önemli Inline Endpoint'ler .....
13.2.8 Arka Plan İzleyiciler .....
13.2.9 Port ve Çalışma Davranışı .....
13.3 Backend Yardımcı Dosyalar (Dosya Bazlı Liste) .....
Bağımlı Modüller: .....
Dosya Adı: socket_listener.js .....
Veri Kayıt Akışı: .....
Dosya Adı: sumo_fiware_bridge.js .....
Çalışma Mekanizması: .....
Dosya Adı: watchRouteSync.js .....
Dosya Adı: hardware.js .....
Dosya Adı: reloadDb.js .....
Dosya Adı: insertMockData.js .....
Dosya Adı: musoshi001.js .....
Dosya Adı: auto_converter.py .....
13.4 Middleware Detayları .....
middleware/metrics.js .....
13.5 Model Dosyaları .....
models/vehicle.js (MariaDB) .....
models/customer.js (MongoDB) .....
models/performance.js (MongoDB) .....
13.6 Route Dosyaları (API Endpoint'leri) .....
routes/vehicles.js .....

routes/route.js .....
routes/orders.js .....
routes/astar.js .....
routes/federated.js .....
13.7 Utility Dosyaları .....
utils/parseRoute4Plan.js .....
14. Frontend Katmanı Genel Mantığı .....
14.1 Frontend-Backend İletişimi .....
API Endpoint Tanımları: .....
14.2 Uygulama Yapısı (App.js) .....
14.3 Fleet Monitoring Ekranı .....
1. Gerçek Zamanlı Araç Takibi .....
2. Uyarı Yönetimi .....
3. Performans İzleme .....
4. A* Rota Hesaplama .....
Araç ID Eşleştirmeleri: .....
14.4 Route Optimization Ekranı .....
14.5 Kimlik Doğrulama Sistemi .....
15. Makine Öğrenmesi ve Tahmin Servisleri .....
15.1 Machine Klasörü Yapısı .....
15.2 app.py - Ana Tahmin Servisi .....
15.3 Federated Learning Mimarisi .....
15.4 Tahmin Sonuçlarının Kullanımı .....
16. Rota ve Simülasyon Servisleri .....
16.1 Route-Service Yapısı .....
16.2 app.py - Rota Servisi .....
16.3 Graf Yönetimi (graph_manager.py) .....
16.4 Edge Tabanlı A* (osmnx_astar_edge_based.py) .....
16.5 SUMO Simülasyon Servisi .....
16.6 main.py - SUMO API Sunucusu .....
16.7 function.py - Simülasyon Mantığı .....
16.8 OR-Tools Optimizasyon (ortools_func.py) .....
16.9 Route4Sim.xml Formatı .....
17. Sağlık İzleme Servisleri .....
17.1 Sistem Genel Bakış .....

17.2 Mimari ve Tasarım
17.3 Ön Koşullar
17.4 Kurulum Kılavuzu
17.5 Config Dosyaları
17.6 Admin Panel Sağlık İzleme Sekmesi (Health Monitoring Tab)
17.7 Alert Tanımları
18. Sistem Geneli Uçtan Uca Veri Akışı
18.1 Veri Girişi - Araç Telemetrisi
18.2 Rota Planlama Akışı
18.3 Simülasyon Çalıştırma
18.4 Tahmin Servisleri Akışı
18.5 A* Rota Hesaplama Akışı
18.6 Uyarı Sistemi Akışı
18.7 Sağlık İzleme Akışı
19. Yeni Geliştiriciler İçin Özet
19.1 Projeye Başlangıç Rehberi
Çalıştırma Sırası:
19.2 Güvenle Müdahale Edilebilecek Dosyalar
19.3 Kritik Dosyalar
19.4 Geliştirmeye Güvenli Başlama Önerileri
1. Yerel Ortam Kurulumu
2. İlk Test
3. Küçük Değişiklikle Başla
4. Loglama Kullan
5. Git Branch Stratejisi
20. Database İşlemleri Localhost için
21. ER Şeması
22. MariaDB Veritabanı
23. MongoDB Veritabanı
23.1 RouteManagementDB
23.1.1 Rota Optimizasyon Planları - route4plans
23.1.2 Araç Enerji Durumu /vehicleEnergy
23.1.3 Sipariş ve Operasyonel Veriler
24. Rotalama Algoritmaları (Diğer)
24.1 Multi-Thread SA-TS-ALNS Servisi
24.2 Heterojen Filolu Rotalama için Algoritma

24.3 Dinamik Rotalama için Algoritma .....
24.4 Rotalama Algoritmalarının Girdi ve Çıktı Standartı - Routing Markup Language (RoutingML) .....
24.4.1 RoutingML'in Amacı ve Tasarım İlkeleri .....
24.4.2 RoutingML Genel Yapısı .....
24.4.3 Girdi Yapısı (Input Specification) .....
24.4.4 Çıktı Yapısı (Output Specification) .....

Bu dokümantasyon, Filo Yönetimi Uygulaması v1.0 hem kod tabanı temel alınarak geliştiriciler için hem de kullanıcı kılavuzu bakımından hazırlanmıştır.

Son güncelleme: 18 Aralık 2025

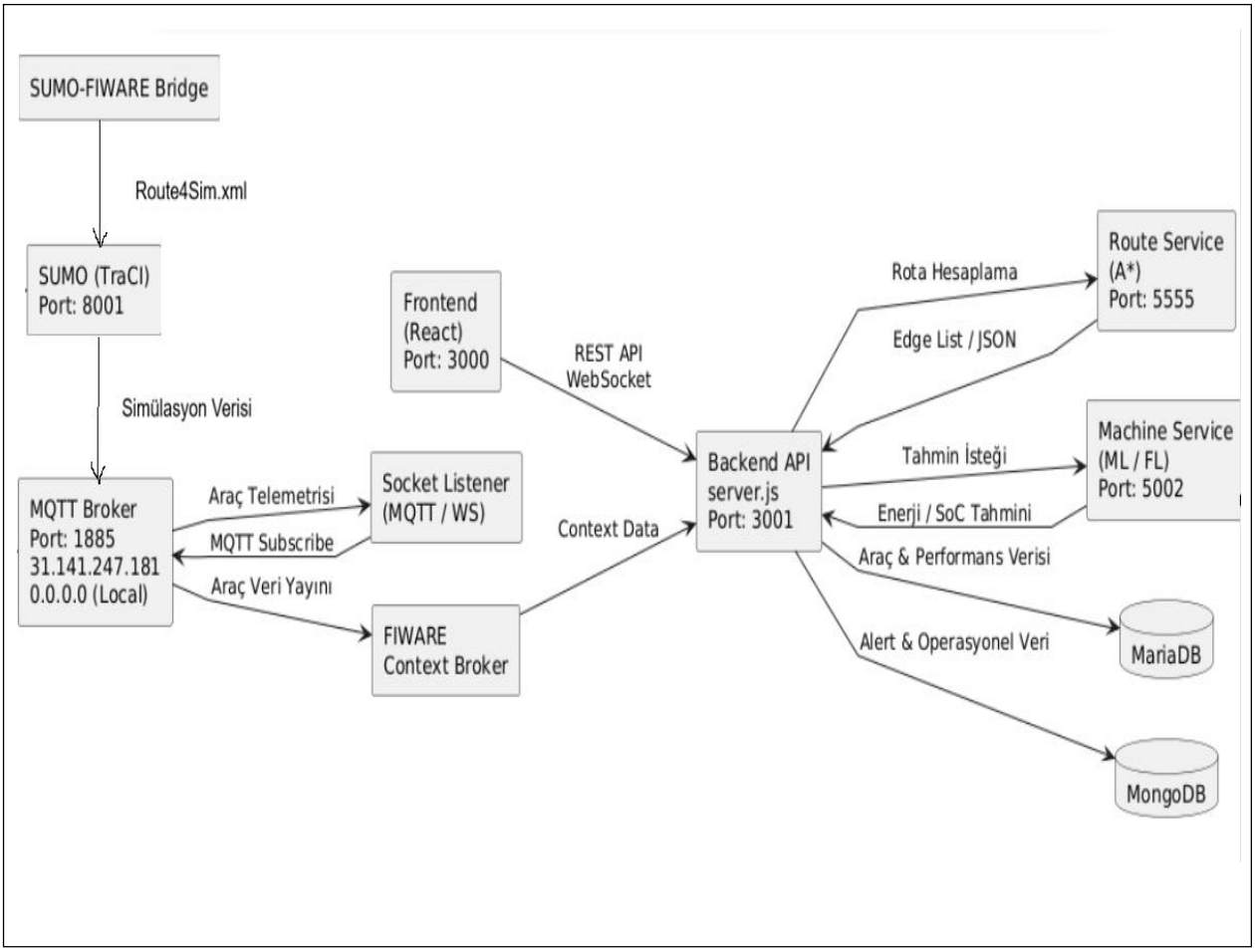
1. Dokümanın Amacı

Bu doküman, filo yönetimi uygulamasının devralınması, kurulumu, temel kullanımı ve operasyonel sürekliliğinin sağlanması amacıyla hazırlanmıştır. Doküman; sistem yöneticileri, geliştiriciler ve teknik destek sağlayacak ekip üyeleri için bir devir teslim ve oryantasyon rehberi niteliğindedir. Tüm süreç ilgili mail dökümünden de takip edilebilir şekilde paylaşılmıştır:
https://drive.google.com/file/d/1MnRK3gl1LEpY7I_HWO1JxzZOEs45NbPb/view?usp=drive_link

2. Sistem Genel Mimarisi (Özet)

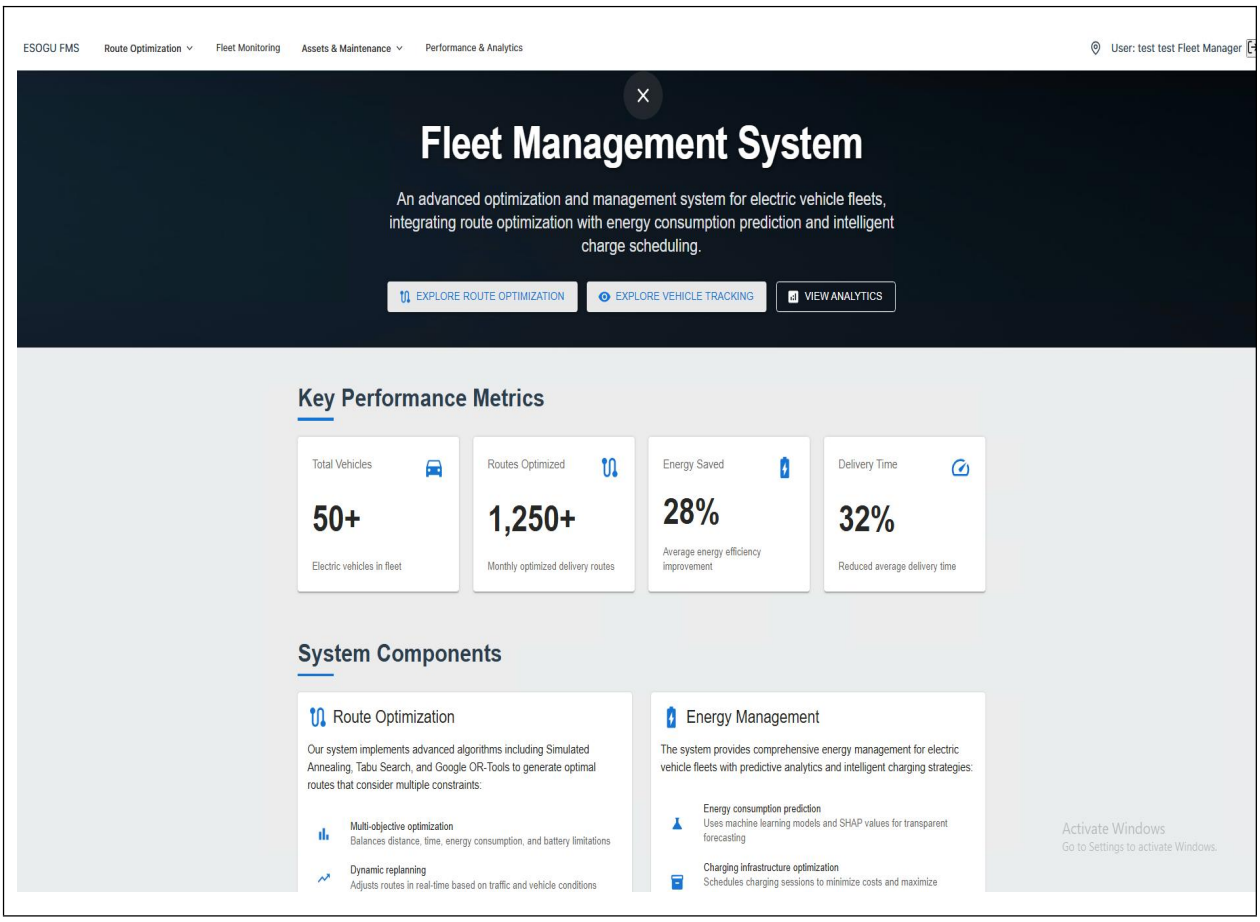
Uygulama aşağıdaki ana bileşenlerden oluşmaktadır: Frontend (React) Backend servisleri (Node.js / Python tabanlı API'ler) Veritabanları (MongoDB Atlas, MariaDB) Simülasyon ve optimizasyon servisleri (SUMO, ALNS, DQN, Dynamic DQN, Q-Learning, A*, DDQN) IoT & veri akışı (FIWARE, MQTT) İzleme ve kalite araçları (Grafana) CI/CD ve kod yönetimi (GitLab, Jira)

Bileşenler Arası Sistem Mimarisi



2.1 Uygulamaya Genel Bakış

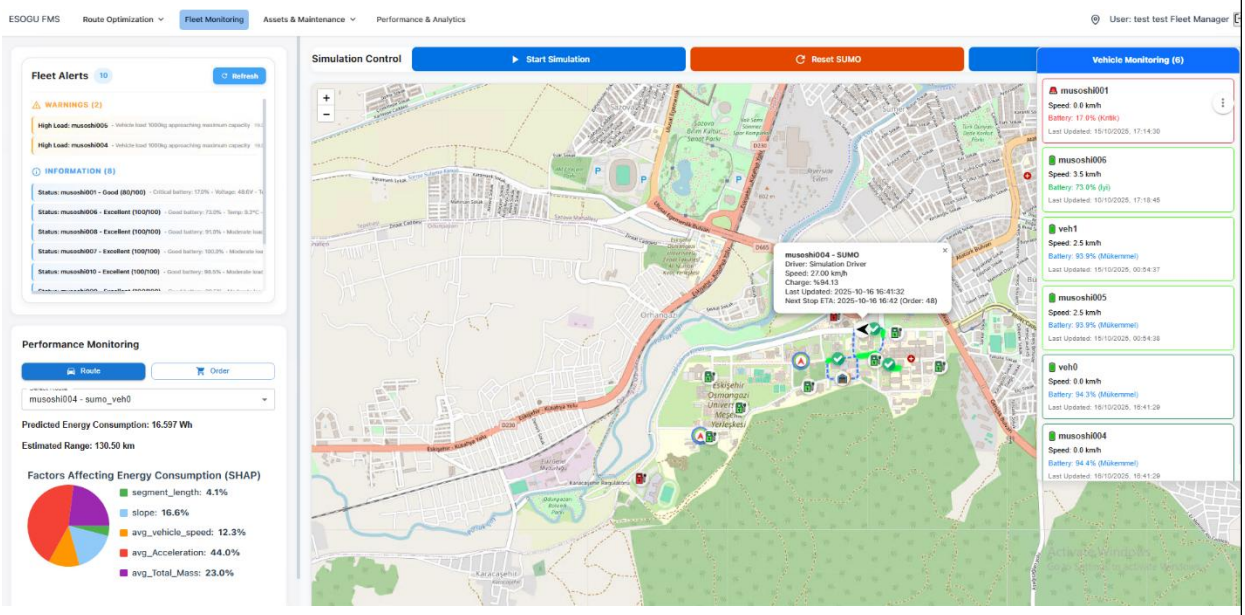
Şekil 1 Anasayfa



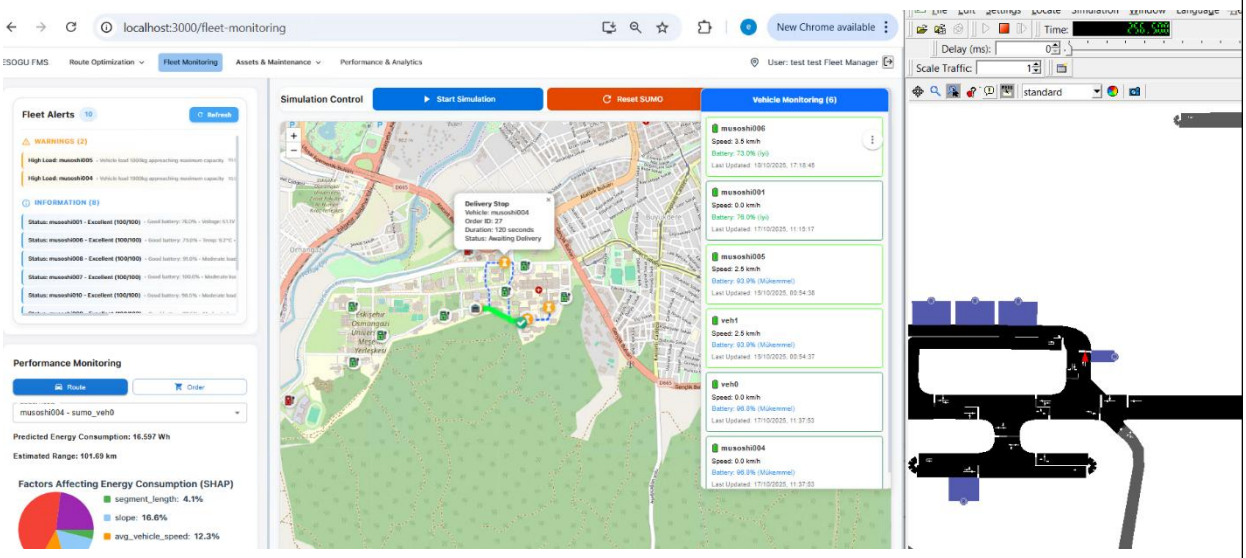
Bu ana sayfa, **Fleet Management System (FMS)** platformunun genel amacını ve sunduğu temel yetenekleri tek bakışta özetleyen bir giriş ekranı olarak tasarlanmıştır. Sayfanın üst bölümünde sistemin, **elektrikli araç filoları için gelişmiş yönetim ve optimizasyon** çözümleri sunduğu vurgulanmakta; rota optimizasyonu, enerji tüketimi tahmini ve akıllı şarj planlamasının bütünleşik biçimde ele alındığı ifade edilmektedir. Orta bölümde yer alan **Key Performance Metrics** alanı, sistemin operasyonel etkisini sayısal göstergelerle ortaya koyarak filo büyüklüğü, optimize edilen rota sayısı, sağlanan enerji tasarrufu ve teslimat süresindeki iyileşmeleri özetlemektedir. Alt bölümde ise **System Components** başlığı altında, rota optimizasyonu ve enerji yönetimi gibi çekirdek modüller açıklanmakta; çok amaçlı optimizasyon, gerçek zamanlı yeniden planlama, enerji tahmini ve şarj altyapısı optimizasyonu gibi yeteneklerle sistemin karar destek ve operasyonel verimlilik odağı ön plana çıkarılmaktadır. Genel olarak ana sayfa, kullanıcıya sistemin kapsamını, katma değerini ve hangi işlevlerle operasyonları desteklediğini sade, ölçülebilir ve profesyonel bir dille sunmaktadır.

2.2 Araç İzleme Modülü (Fleet Monitoring)

Şekil 2 Araç İzleme



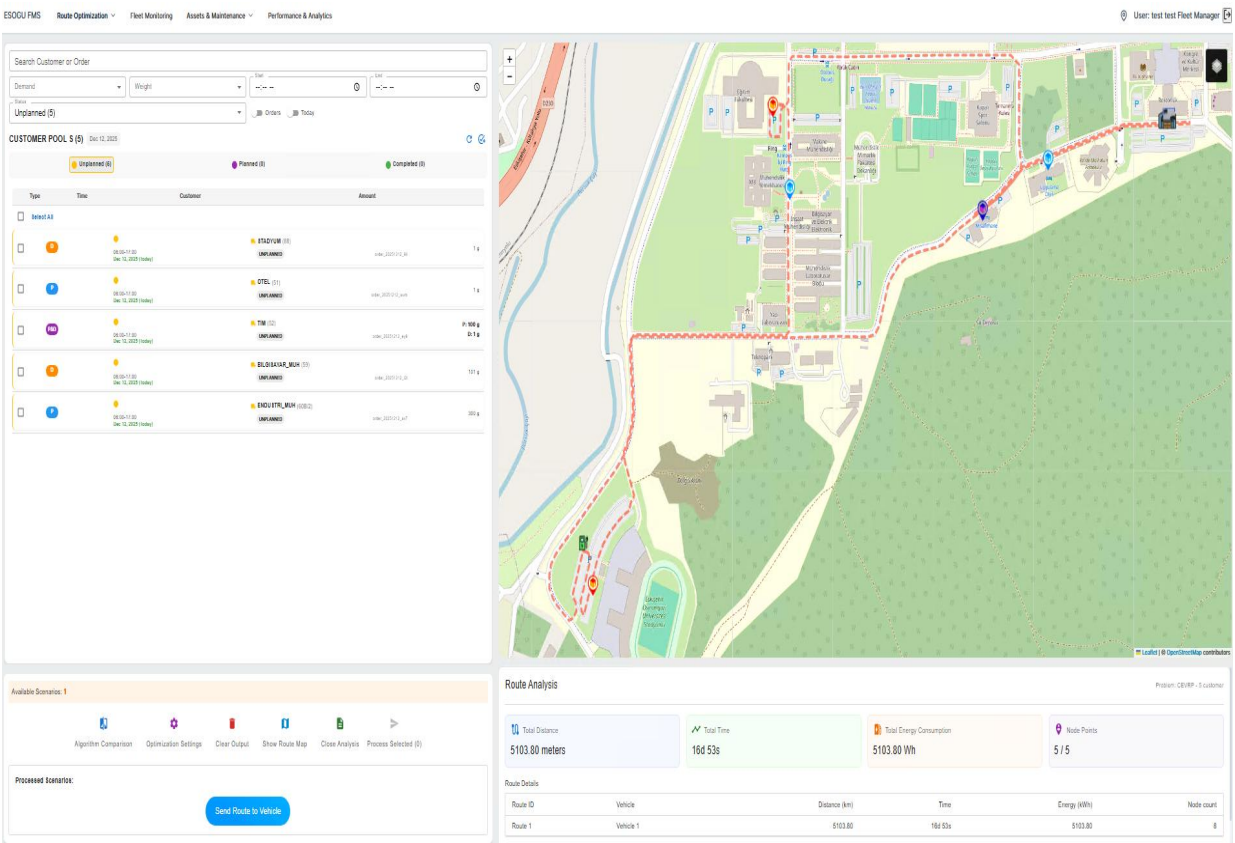
Şekil 3 Sumo ile Birlikte Gösterim



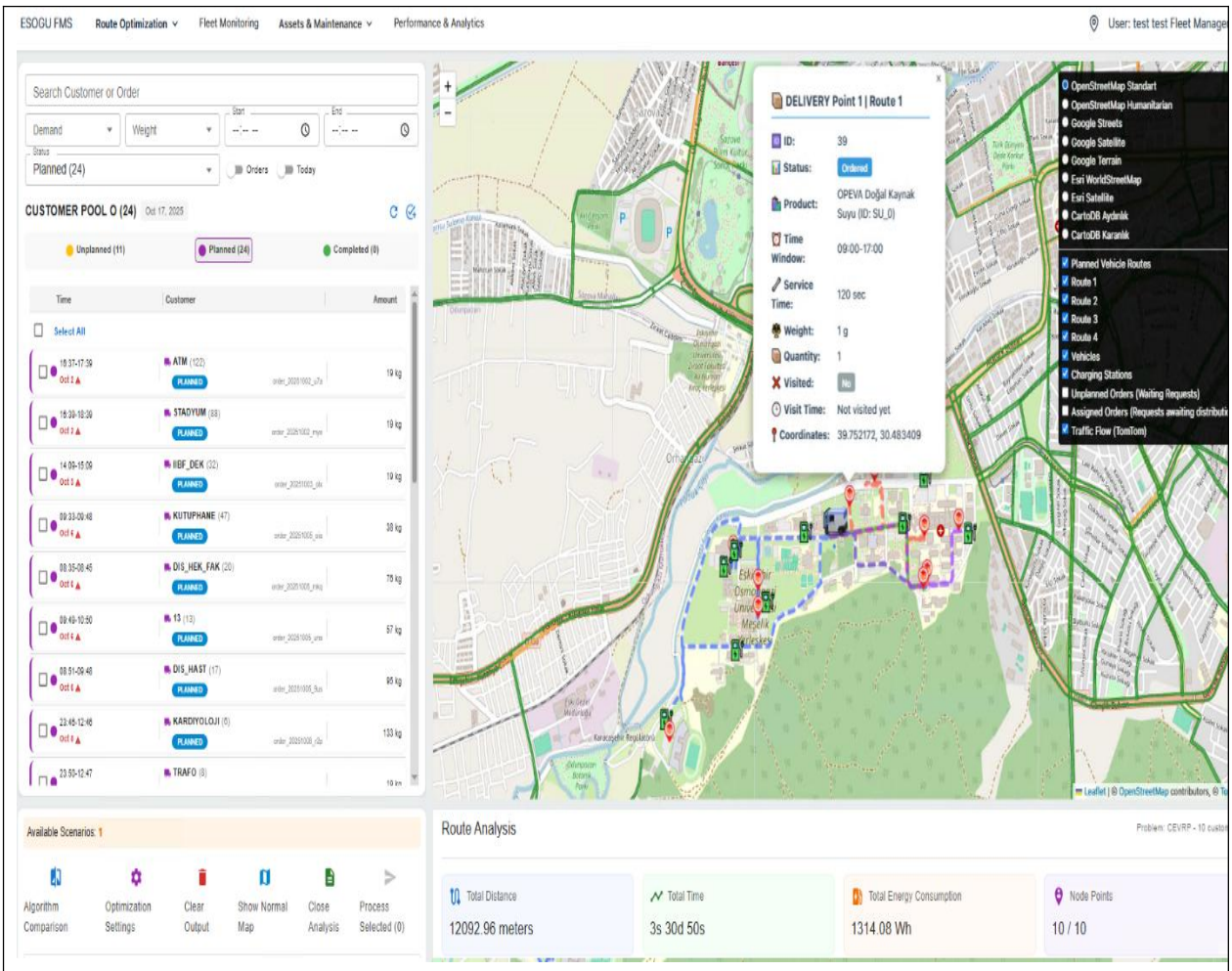
Bu ekran, **Fleet Monitoring** modülü kapsamında elektrikli araç filosunun **gerçek zamanlı izlenmesi ve simülasyon tabanlı kontrolünü** sağlayan merkezi bir operasyon paneli olarak tasarlanmıştır. Harita üzerinde SUMO simülasyonu ile entegre şekilde araçların anlık konumları, hızları, batarya seviyeleri ve tahmini varış süreleri görselleştirilirken; üst bölümde yer alan **Simulation Control** alanı üzerinden simülasyonun başlatılması veya sıfırlanması mümkün olmaktadır. Sol panelde bulunan **Fleet Alerts** bölümü, yüksek yük, düşük batarya veya operasyonel riskler gibi durumları uyarı ve bilgilendirme seviyelerine ayırarak filo yöneticisine hızlı aksiyon alma imkânı sunar. Alt kısımda yer alan **Performance Monitoring** alanı ise seçili araç veya rota için tahmini enerji tüketimi, menzil bilgisi ve **SHAP tabanlı açıklanabilir yapay zekâ** çıktıları ile enerji tüketimini etkileyen faktörleri analitik olarak ortaya koymaktadır. Sağ paneldeki **Vehicle Monitoring** listesi sayesinde her bir aracın durumu (hız, batarya durumu, güncelleme zamanı) anlık olarak izlenebilirken, sistem genel olarak filo operasyonlarının hem operasyonel hem de analitik açıdan uçtan uca yönetilmesini sağlayan bütünleşik bir karar destek arayüzü sunmaktadır.

2.3 Rota Optimizasyonu Modülü (Route Optimization)

Şekil 4 Rota Optimizasyonu



Route Optimization S

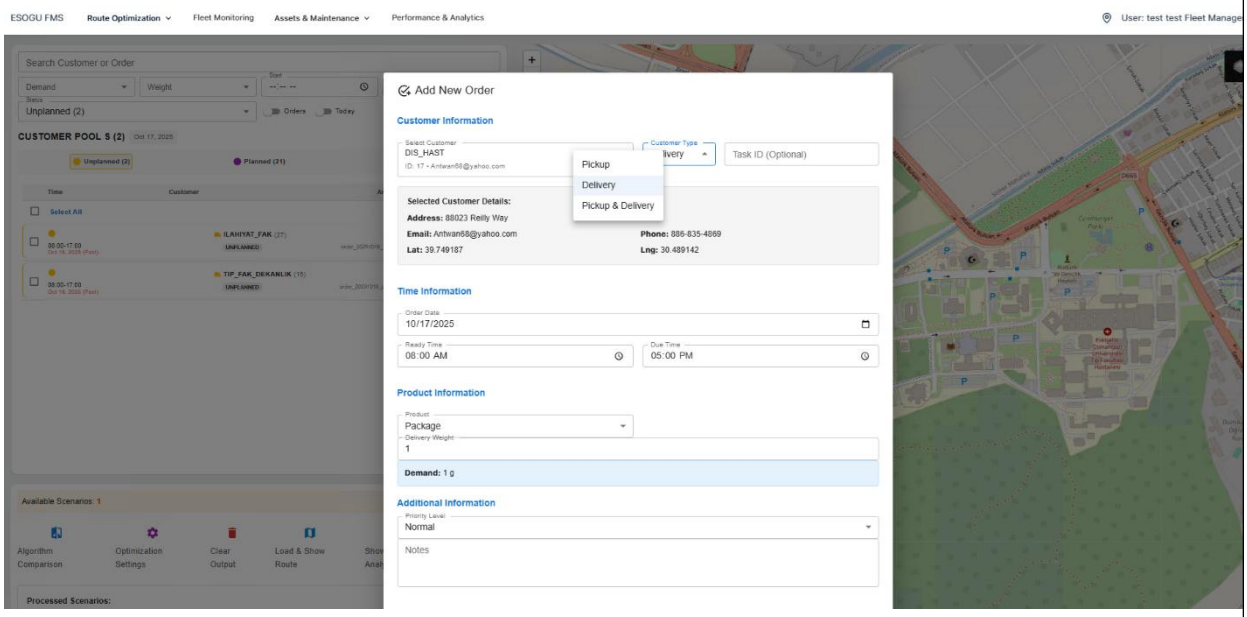


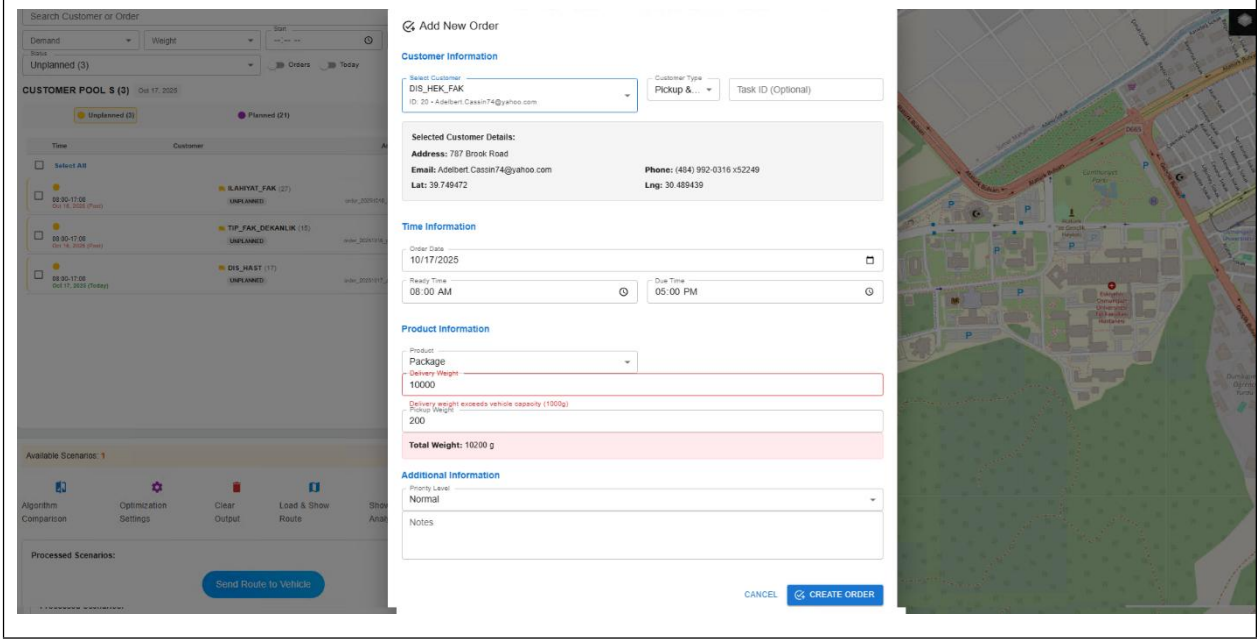
Route Optimization O

Bu ekran, **Route Optimization** modülü kapsamında müşteri taleplerinin planlanması, rotaların oluşturulması ve sonuçların analitik olarak değerlendirilmesini sağlayan bütünleşik bir çalışma alanı sunmaktadır. Sol tarafta yer alan **Customer Pool** bölümü üzerinden siparişler; durumlarına göre (planlanmamış, planlanmış, tamamlanmış) filtrelenebilmekte, talep tipi, zaman aralığı ve ağırlık gibi kriterlere göre seçilerek optimizasyona dâhil edilebilmektedir. Orta bölümde yer alan harita alanı, seçilen müşteri noktalarının ve oluşturulan rotanın coğrafi dağılımını görsel olarak sunarak güzergâhın sezgisel biçimde değerlendirilmesine imkân tanımaktadır. Alt bölümde bulunan **Route Analysis** alanı, toplam mesafe, süre, enerji tüketimi ve kapsanan düğüm sayısı gibi temel performans göstergelerini özetleyerek çözümün operasyonel verimliliğini ortaya koymaktadır. Ayrıca senaryo yönetimi, algoritma karşılaştırması ve optimizasyon ayarları gibi kontroller sayesinde farklı çözüm yaklaşımlarının test edilmesi mümkün olmakta; elde edilen rotalar **Send Route to Vehicle** fonksiyonu ile doğrudan araçlara iletilerek planlama ile saha operasyonları arasında kesintisiz bir entegrasyon sağlanmaktadır.

2.3.1 Müşteri Ekleme

Şekil 5 Havuza Müşteri Ekleme



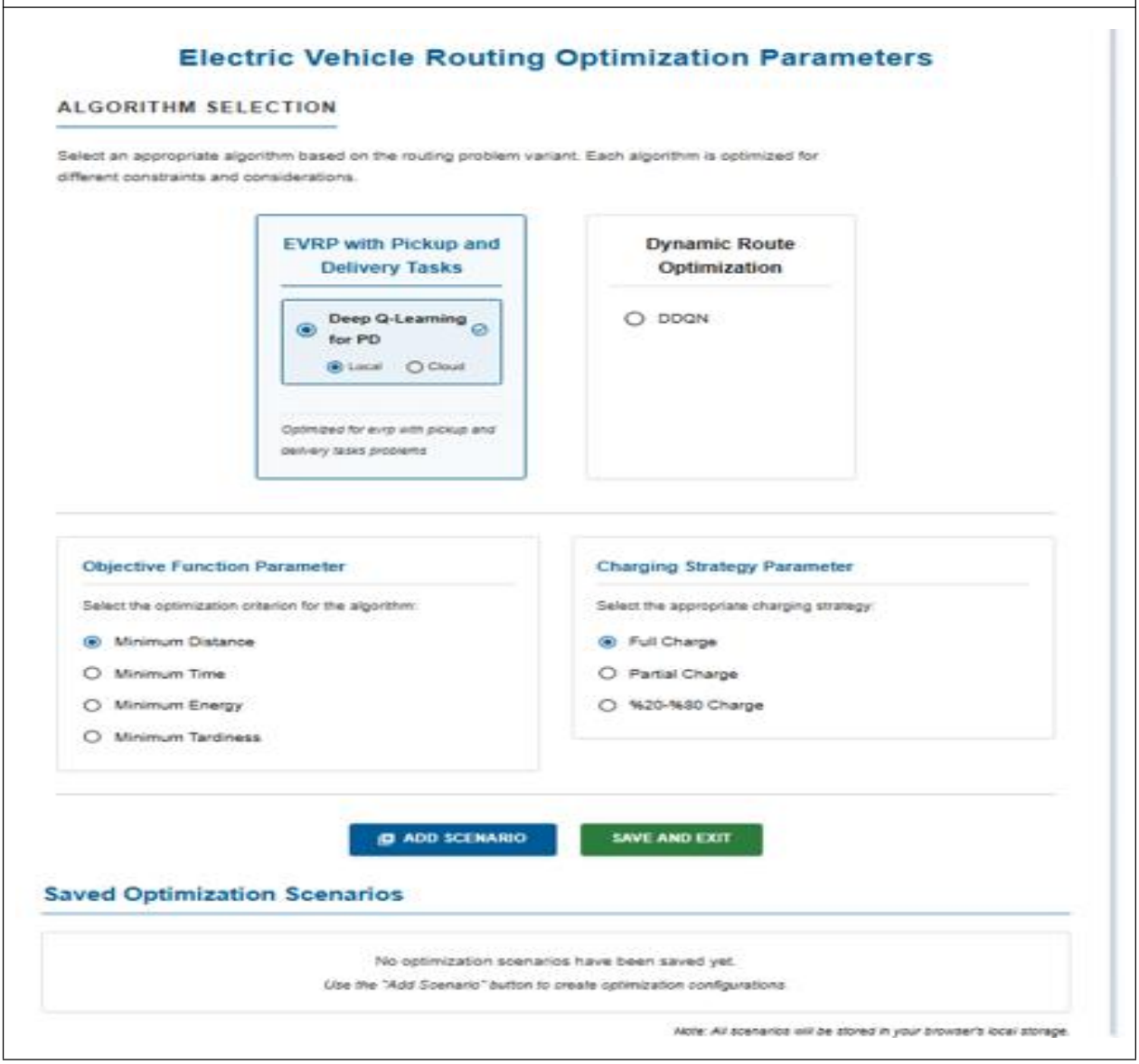


Bu ekran, **Customer Pool (Müşteri Havuzu)** içerisine yeni bir müşteri/sipariş ekleme sürecini ayrıntılı ve kontrollü bir şekilde yöneten **Add New Order** formunu göstermektedir. Süreç, öncelikle **Customer Information** alanında mevcut bir müşterinin seçilmesiyle başlar; seçilen müşteriye ait adres, e-posta, telefon ve konum bilgileri otomatik olarak doldurularak veri tutarlılığı sağlanır. Ardından **Time Information**

bölümünde sipariş tarihi, hazır olma zamanı ve teslimat için son zaman (due time) tanımlanarak zaman penceresi kısıtları belirlenir. **Product Information** alanında ürün tipi ile teslimat ve alım ağırlıkları girilir; sistem bu aşamada araç kapasitesini otomatik olarak kontrol eder ve kapasite aşımlarında kullanıcıyı uyararak hatalı siparişlerin havuza eklenmesini engeller. Ek olarak **Additional Information** bölümünde öncelik seviyesi ve açıklama notları tanımlanabilir. Tüm kontroller tamamlandığında sipariş, **Create Order** butonu ile müşteri havuzuna eklenir ve daha sonraki rota optimizasyonu süreçlerinde planlanmaya hazır hâle getirilir. Bu yapı sayesinde müşteri ekleme süreci hem kullanıcı dostu hem de operasyonel kısıtları gözeten güvenli bir biçimde yönetilmektedir.

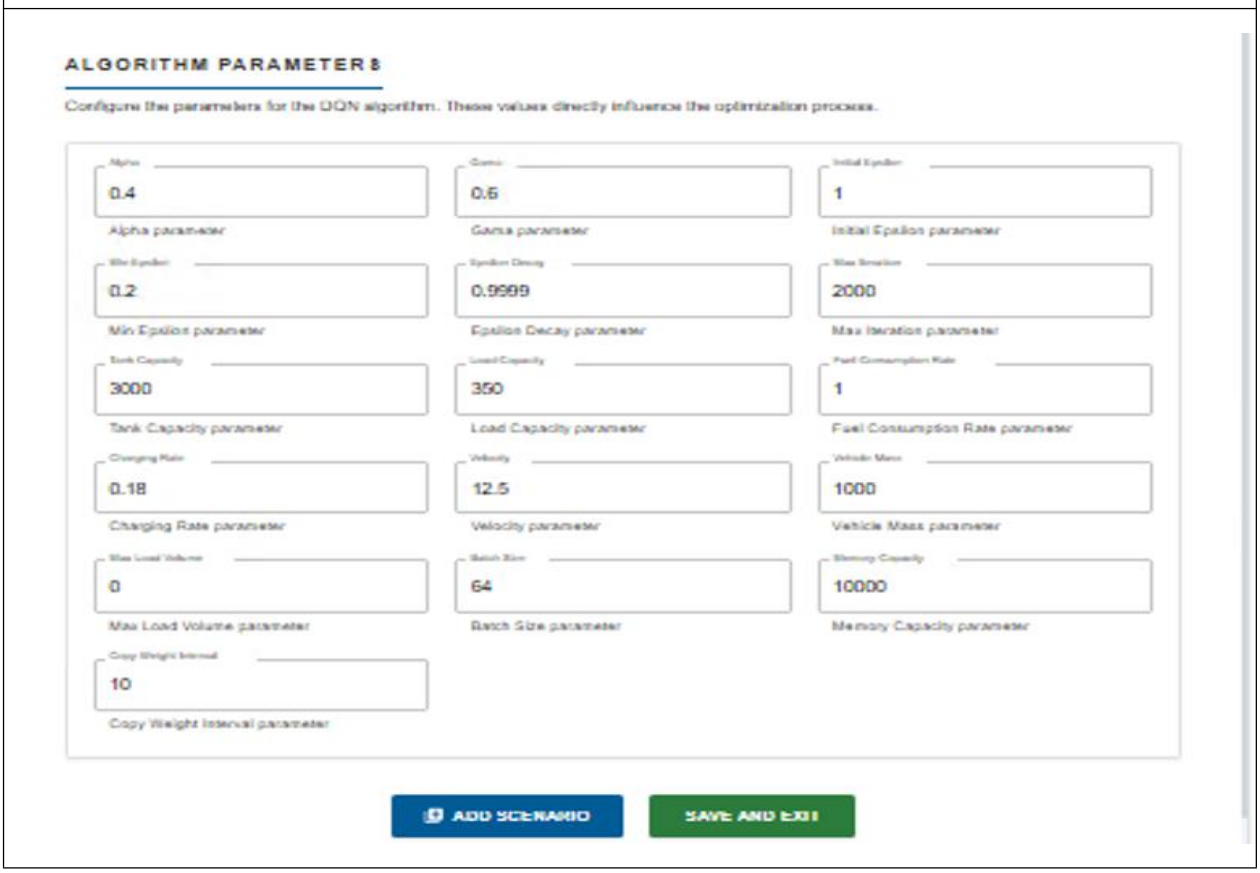
2.3.2 Algoritma Parametre Giriş ve Algoritma Seçim Ekranı

Şekil 6 Parametreler ve Algoritma Seçim Ekranı



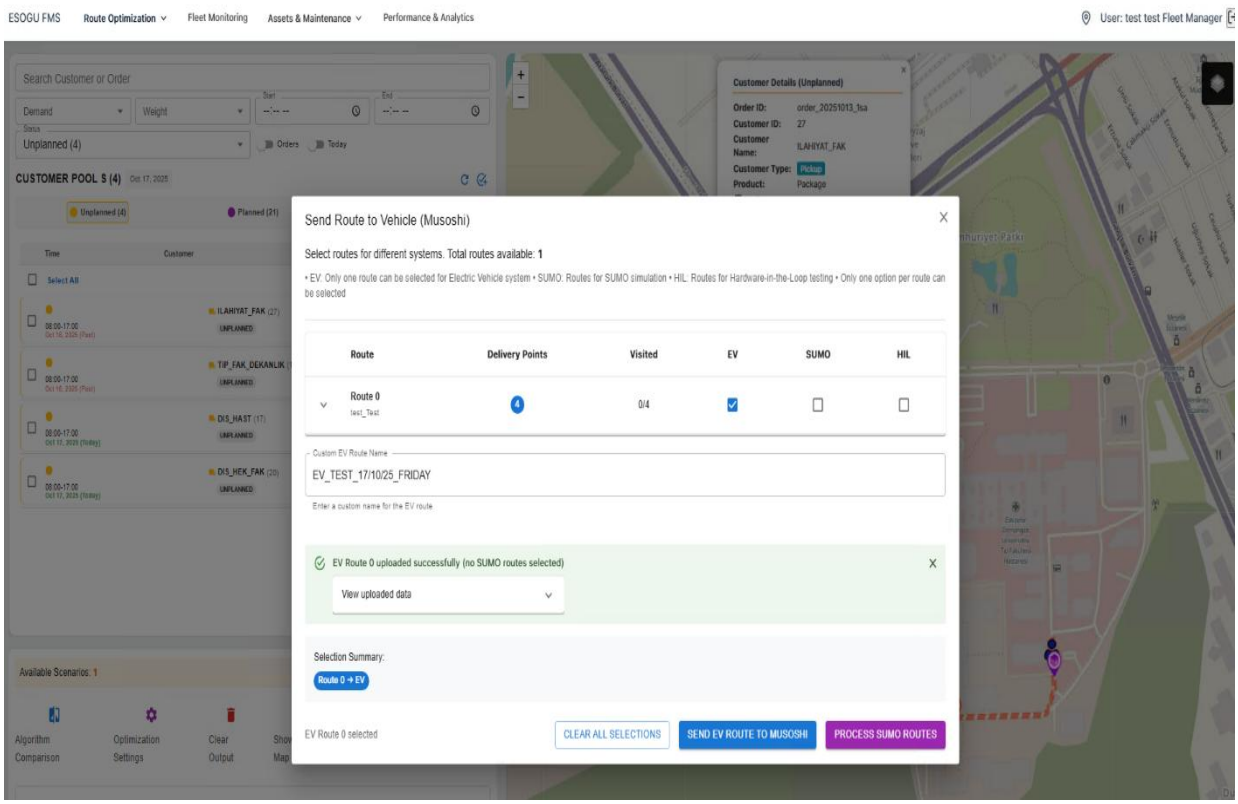
Bu ekran, Electric Vehicle Routing Optimization Parameters başlığı altında elektrikli araçlar için rota optimizasyonunun nasıl gerçekleştirileceğini tanımlayan senaryo yapılandırma panelini göstermektedir. Panelin üst bölümünde yer alan Algorithm Selection alanı sayesinde kullanıcı, çözülmek istenen problem tipine uygun algoritmayı seçebilmektedir. Burada, alım–teslimat görevlerini içeren EVRP senaryoları için Deep Q-Learning tabanlı yöntem veya dinamik koşullara uyum sağlayan Dynamic Route Optimization (DDQN) yaklaşımı tercih edilebilmektedir; ayrıca çözümün yerel veya bulut ortamında çalıştırılması seçeneği sunulmaktadır.Orta bölümde bulunan Objective Function Parameter alanı, optimizasyonun hangi hedef doğrultusunda yapılacağını belirlemeye imkân tanır; kullanıcı toplam mesafenin, sürenin, enerji tüketiminin veya gecikmenin minimize edilmesini önceliklendirebilir. Bununla birlikte Charging Strategy Parameter bölümü, araçların rota boyunca hangi şarj stratejisini uygulayacağını tanımlayarak tam şarj, kısmi şarj veya belirli bir şarj aralığında (örneğin %20–%80) çalışma gibi seçenekler sunar. Alt bölümde ise oluşturulan konfigürasyonların senaryo olarak kaydedilmesine ve daha sonra tekrar kullanılmasına olanak tanınmaktadır. Add Scenario ve Save and Exit kontrolleri sayesinde farklı algoritma ve hedef kombinasyonları kolayca karşılaştırılabilir hâle gelmekte; böylece filo yöneticileri, operasyonel koşullara en uygun optimizasyon stratejisini sistematik ve esnek bir biçimde belirleyebilmektedir.

Şekil 7 Algoritma Parametre Giriş Özelliği



2.3.3 Send to Route (EV, HIL, SUMO)

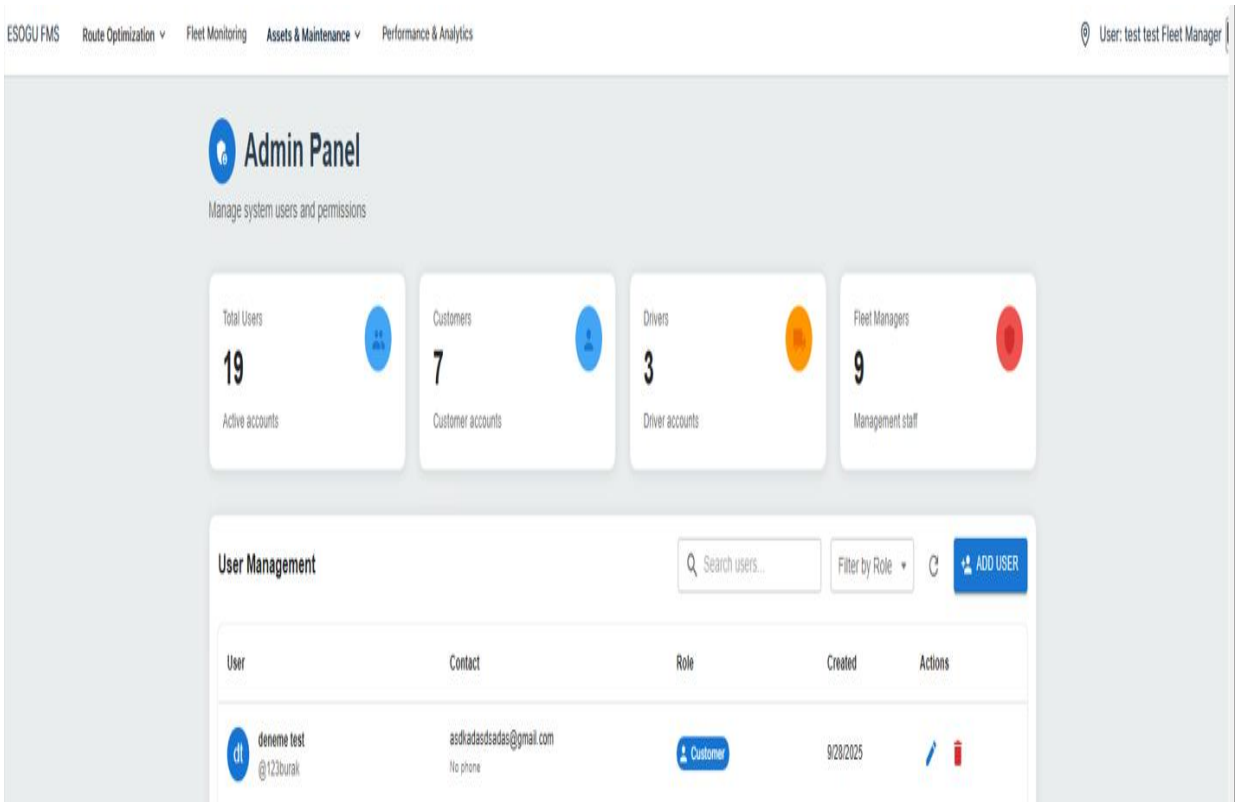
Şekil 8 Rota Gönderim



Bu ekran, **Route Optimization** süreci sonucunda elde edilen rotaların **araçlara, simülasyon ortamlarına veya donanım test sistemlerine gönderilmesini** sağlayan **Send Route to Vehicle** arayüzünü göstermektedir. Panelde, oluşturulan rotalar liste halinde sunulmakta ve her bir rota için hangi ortamda kullanılacağı ayrı ayrı seçilebilmektedir. Kullanıcı, rotayı doğrudan **Elektrikli Araç (EV)** sistemine gönderebilir, **SUMO** simülasyonu için işleyebilir veya **HIL (Hardware-in-the-Loop)** test senaryolarında kullanmak üzere işaretleyebilir; her rota için yalnızca bir hedef sistem seçilmesine izin verilerek veri tutarlılığı sağlanmaktadır. Arayüzde ayrıca, seçilen rota için **özel bir rota adı** tanımlanabilmekte ve bu adlandırma sayesinde saha operasyonları, testler ve raporlama süreçleri kolayca izlenebilmektedir. Sistem, seçimin başarıyla tamamlanması durumunda kullanıcıya bilgilendirici bir geri bildirim sunarak rotanın ilgili hedef sisteme yüklendiğini doğrular. Alt bölümde yer alan özet alanı, hangi rotanın hangi sisteme gönderildiğini net bir şekilde gösterirken; **Send EV Route to Musoshi** ve **Process SUMO Routes** butonları sayesinde planlama çıktıları ile gerçek araçlar, simülasyon ve test ortamları arasında **uçtan uca ve kontrollü bir entegrasyon** sağlanmaktadır.

2.3.4 Admin Yönetim Paneli

Şekil 9 Panel



Bu ekran, **ESOGU FMS** platformunun **Admin Panel** bölümünü göstermekte olup sistemde yer alan kullanıcıların ve yetkilerin merkezi olarak yönetildiği idari arayüzü temsil etmektedir. Üst kısımda yer alan özet kartlar aracılığıyla sistemdeki toplam aktif kullanıcı sayısı ile birlikte **müşteri**, **sürücü** ve **filo yöneticisi** rollerine göre dağılım net bir şekilde sunulmaktadır. Bu özet yapı, yöneticilerin kullanıcı ekosisteminin genel durumunu hızlıca değerlendirmesine olanak tanır. Alt bölümde bulunan **User Management** alanı ise kullanıcıların ayrıntılı olarak listelendiği ve yönetildiği ana çalışma alanıdır. Burada her kullanıcı için ad–kullanıcı adı bilgisi, iletişim bilgileri, atanmış rolü ve hesap oluşturulma tarihi görüntülenmektedir. Arama, role göre filtreleme ve yenileme kontrolleri sayesinde büyük kullanıcı listeleri kolayca yönetilebilirken, **Add User** butonu ile sisteme yeni kullanıcılar eklenebilmektedir. Her bir kullanıcı satırında yer alan düzenleme ve silme aksiyonları sayesinde kullanıcı rolleri güncellenebilmekte veya hesaplar kontrollü biçimde sistemden kaldırılabilmektedir.

Admin panel içerisinde bulunan, health monitoing kısmında ise aktif uyarıları, uyarı özetlerini, geçmiş uyarı kayıtlarını gözlemleyebilmenin ve yönetebilmenin yanı sıra, her bir sistemin gerekli grafiklerinin gözlemlenebilmesine ve uyarı kurallarını dinamik olarak değiştirilebilmesine olanak sağlar.

3. Hesaplar ve Yetkilendirme

3.1 MongoDB Atlas

Ana hesap: opevafleetmanagement@gmail.com
Şifre: FMapplication_1
Tüm bireysel ve dışa açık mail erişimleri kaldırılmıştır.
Mevcut collection ve veritabanı yapıları korunmuştur.

Connection String (.env içinde kullanılır):

mongodb+srv://fleetmanager:oQKOn0EFYeUQ6rmF@cluster0.on8m7.mongodb.net/RouteManagementDB?retryWrites=true&w=majority

3.2 Jira / Atlassian

- **Admin hesap:** opevafleetmanagement@gmail.com
- **Şifre:** FMapplication_1
- Sprint açma, üye daveti ve yetkilendirme işlemleri bu hesap üzerinden yapılır.
- Kullanıcı limiti: **10 kişi**
- Yönetim paneli: https://admin.atlassian.com/

4. Ağ ve Sunucu Bilgileri

4.1 Statik IP

- **Sunucu Statik IP:** 31.141.247.181

4.2 Lokal Ağ (ESOGÜ Superbox)

- Modem arayüzü: http://192.168.8.1/html/index.html
- **Şifre:** esogusuperbox1
- *Devices* sekmesinden bağlı cihazlar görüntülenir.

5. Kullanılan Portlar

Aşağıda sistemde aktif olarak kullanılan başlıca portlar listelenmiştir:

Servis	Port	IP
Frontend (Client)	3000	192.168.8.108
Backend API	3001	192.168.8.108
SUMO	8000	192.168.8.108
Socket Server	8035	192.168.8.108
MQTT	1885	192.168.8.108
GitLab	8080	192.168.8.108
Grafana	3004	192.168.8.108
PostgreSQL	5432	192.168.8.108
SonarQube	9000	192.168.8.156

Not: Diğer ALNS, DQN, Dynamic DQN ve test servisleri için ek portlar sistemde tanımlıdır.

6. Firewall (Windows) Port Açma Örneği

Aşağıdaki örnek PowerShell komutları **yönetici (Administrator)** yetkisi ile çalıştırılmalıdır:

```
New-NetFirewallRule -DisplayName "GitLab_8080_TCP" -Direction Inbound -Profile Any -Action Allow -LocalPort 8080 -Protocol TCP
New-NetFirewallRule -DisplayName "GitLab_8080_UDP" -Direction Inbound -Profile Any -Action Allow -LocalPort 8080 -Protocol UDP
```

7. Jetson Nano – FIWARE IP Güncelleme Adımları

FIWARE'a veri gönderimi için Jetson Nano üzerinde IP güncellemesi yapılmalıdır.

1. **SUIT SUPERBOX ağına bağlanın**
2. SSH bağlantısı kurun: ssh jetson-nano-io@192.168.8.136 Şifre: 1
3. Proje dizinine gidin: cd Desktop/OPEVA_sv34
4. config.sh dosyasını açın: sudo micro config.sh ardından 8.108 ile biten eski IP adresini güncel IP ile değiştirin.
5. .env dosyasını açın: sudo micro .env ardından SOCKET_IP değerini güncelleyin (port aynı kalır).
6. Docker servislerini yeniden başlatın:
7. sudo docker-compose down
   sudo docker-compose up -d

8. Kod ve Yedekler

Tüm kurulum dosyaları, kodlar ve yedekler aşağıdaki dizinde toplanmıştır:

2025_DEVIR_TESLIM_ARALIK_UYGULAMA
https://drive.google.com/drive/u/1/folders/1HEeDgRFBRNCAEGG9LNzSYo0wmIFEKf2M

Öneri: Klasörün lokal ve harici bir ortamda yedeği alınmalıdır.

9. GitLab CI/CD Süreçleri Hak.

Gitlab-usage ve gitlab-membership pdf üzerinde süreç hakkında işlemler detaylı olarak verilmiştir.

https://drive.google.com/file/d/1Sf-jMT354F_IEbecqvtPlbNJmLXTJMoj/view?usp=drive_link

https://drive.google.com/file/d/1lnJa883ECNXMoyVAdw1mCc5hRJ6cvMOO/view?usp=drive_link

GitLab 1004 sunucusu üzerinde docker' da kuruludur. Localhost:8080 üzerinden erişim sağlanabilir.

Id: root Password: myLab7979

Admin paneline erişim sağlanarak CI/CD süreçleri yönetilebilir, pull/push işlemleri “revize005” branchi açılarak üzerinde gerçekleştirilmiştir. Main repository güvenli moddadır.

10. Fiware Ip Configuration

Fiware ip configuration pdf üzerinde süreç detaylı olarak anlatılmıştır.

https://drive.google.com/file/d/18fBPfOtNrUybJAlFjSdnZMaWjILbp7ML/view?usp=drive_link

11. Uygulama Kurulum ve Servis Başlatma Adımları

Bu bölümde, filo yönetimi uygulamasının lokal veya yarı-üretim ortamında **uçtan uca kurulumu**, servislerin başlatılması ve bileşenler arası veri akışının sağlanması adım adım açıklanmaktadır.

11.1 Veritabanı ve Gerekli Yazılımlar

Aşağıdaki temel bileşenler kurulu olmalıdır:

- **MariaDB** kurulmuş ve varsayılan port olan **3306** üzerinden çalışmaktadır.
- **HeidiSQL** kurulmuş ve MariaDB yönetimi için kullanılmaktadır.
- **Node.js** kurulmuştur (backend ve frontend bağımlılıkları için gereklidir).
- **MongoDB Compass** kurulmuştur (MongoDB Atlas veritabanlarını IDE üzerinden yönetmek için).

MariaDB Veri Aktarımı ve Kullanıcı Yetkilendirme

- fleetmanagementdb.sql dosyası HeidiSQL üzerinden içe aktarılmıştır.
- Uzaktan erişim ve uygulama bağlantıları için aşağıdaki kullanıcı oluşturulmuştur:

```
CREATE USER 'remoteuser'@'%' IDENTIFIED BY 'arm2028';
GRANT ALL PRIVILEGES ON *.* TO 'remoteuser'@'%' WITH GRANT OPTION;
FLUSH PRIVILEGES;
```

- remoteuser kullanıcısı tüm veritabanları üzerinde tam yetkilidir.

11.2 Backend Servisi Kurulumu

Backend dizinine geçilerek Node.js bağımlılıkları kurulmalıdır:

cd C:\Users\S\Desktop\Application\app\app\backend

npm install

npm install bcryptjs

Bu işlem sonrasında backend servisleri çalıştırılabilir duruma gelir.

11.3 Frontend Servisi Kurulumu ve Çalıştırılması

Frontend uygulaması aşağıdaki adımlar ile başlatılır: Frontend servis varsayılan olarak **3000** portu üzerinden çalışmaktadır.

cd C:\Users\S\Desktop\Application\app\app\frontend

npm install ile kütüphaneler yüklenir sonra başlatmak için npm start

11.4 A* Route Service Kurulumu

A* route servisi Python sanal ortamı üzerinden çalışmaktadır:

cd C:\Users\S\Desktop\Application\app\app\route-service

python -m venv .venv

pip install -r requirements.txt

Kurulum sonrası servis çalıştırılabilir.

11.5 SUMO Simülasyon Servisi

SUMO tabanlı simülasyon servisi için ayrı bir sanal ortam oluşturulmuştur:

cd C:\Users\S\Desktop\Application\app\app\Sumo

python -m venv .venv

pip install -r requirements.txt

python main.py

Bu servis, simülasyon tabanlı araç rota ve araç hareketlerini üretir.

11.6 Machine Prediction (Tahmin Servisi)

Makine öğrenmesi tabanlı menzil ve tahmin servisi aşağıdaki şekilde çalıştırılır:

```
cd C:\Users\S\Desktop\Application\app\app\machine

python -m venv .venv

.\.venv\Scripts\Activate

pip install -r requirements.txt

python app.py
```

Servis, önceden eğitilmiş modeller (best_model.pkl, global_model.pt) üzerinden tahmin üretmektedir.

11.7 Rota Aktarımı ve Fleet Monitoring Entegrasyonu

Rotaların Fleet Monitoring arayüzüne aktarılabilmesi için backend altında aşağıdaki dosyalarda **lokal path düzenlemesi** yapılmalıdır:

- sumo_fiware_bridge.js
- watchRouteSync.js

Örnek path:

C:/Users/S/Desktop/Application/app/app/Sumo/ESOGU_SUMO/Route4Sim.xml

Rota senkronizasyonunu başlatmak için:

```
cd backend

node sumo_fiware_bridge.js
```

Bu işlem sonrası rota ve araç isim mapping işlemleri Fleet Monitoring arayüzüne yansır.

11.8 Socket Server (8035) – Docker Kurulumu

Lokal ortamda FIWARE veri alışverişlerini dinleyebilmek için socket server Docker üzerinden kurulmaktadır:

```
cd C:\Users\S\Desktop\Application\fiware_socket_server\fiware_socket_server

docker network create opeva

docker-compose up --build -d
```

Bu işlem sonrası socket server **8035** portu üzerinden aktif hale gelir.

11.9 Algoritma Servisleri

Sistemde kullanılan algoritmalar ve port bilgileri aşağıdaki gibidir:

- **ALNS:** 8005 (Local)

- **DQN:** 8006
- **Q-Learning:** 8007
- **DQN-PD:** 8008
- **DDQN:** 8009

Örnek çalıştırma komutu:

python -m uvicorn server:app --host 0.0.0.0 --port 8006

11.10 FIWARE Servisleri

FIWARE bileşenleri Docker üzerinden aşağıdaki komutlar ile ayağa kaldırılır:

cd fiware_v2/fiware_components

docker-compose up --build -d

cd fiware_v2/fiware_mqtt_server

docker-compose up –build

cd file_collection_service_v1

docker-compose up --build -d

Not: Dockerfile tabanı olarak **python:3.10-slim-bookworm** kullanılmaktadır. Önceki slim imajlar kütüphane uyumsuzluğu nedeniyle güncellenmiştir.

11.11 FIWARE Components Olası Arıza Durumu Hakkında

Olası durma/çalışmama durumlarına karşın Fiware components docker üzerinde durdurulup, silinerek yeniden build yapılmalıdır.

cd fiware_v2/fiware_components

docker-compose up --build -d

12. Kod Tabanı Genel Yapısı

12.1 Proje Mimarisi Genel Bakış

Filo Yönetimi Uygulaması, mikro servis mimarisine yakın bir yapıda tasarlanmış, çok katmanlı bir sistemdir. Uygulama beş ana bileşenden oluşmaktadır:

Backend | Node.js / Express | 3001 | API Gateway, Veritabanı İşlemleri, MQTT/Socket İletişimi

Frontend | React.js | 3000 | Kullanıcı Arayüzü, Harita Görselleştirme

Machine | Python / Flask | 5002 | ML Tahmin Servisleri, Federated Learning

Route-Service | Python / Flask | 5555 | A* Rota Hesaplama, OSMnx Entegrasyonu

Sumo | Python / Flask + TraCI | 8001 | Trafik Simülasyonu, Araç Hareketleri

12.2 Veri Akış Özeti

1. **Gerçek Araç Verisi**: MQTT Broker (1885) üzerinden FIWARE protokolüyle alınır

2. **Simülasyon Verisi**: SUMO TraCI arayüzü üzerinden anlık olarak okunur

3. **Rota Optimizasyonu**: Frontend tarafından talep edilir, arayüzden seçilen müşteri verisi ilgili algoritmaya etc. ALNS gönderilir, çözüm xml formatında frontend output rml dosyasında xml formatında tutulur. A* ise route-service ile hesaplanır.

4. **Tahmin Servisleri**: Machine modülü ML/FL modelleriyle enerji/SoC tahmini yapar

5. **Görselleştirme**: Frontend tüm verileri harita ve grafiklerle sunar

12.4 Dosya Referansları

SİSTEM BAŞLATMA (start_all.bat)

#	Proses Adı	Komut	Port	Açıklama
1	backend:server	node server.js	3001	Ana API sunucusu
2	backend:bridge	node sumo_fiware_bridge.js	-	SUMO FIWARE köprüsü

#	Proses Adı	Komut	Port	Açıklama
3	backend:watch	node watchRouteSync.js	-	XML dosya senkronizasyonu
4	sumo	py main.py	8001	SUMO simülasyon API
5	machine	py app.py	5002	ML tahmin servisi
6	frontend	npm start	3000	React web arayüzü
7	route-service	py app.py	5555	A* rota hesaplama servisi

BACKEND KLASÖRÜ

Dosya	Satır	Çalışma Şekli	Açıklama
server.js	4074	node server.js	Ana API sunucusu, Express.js, tüm route'ları yükler
socket_listener.js	676	server.js içinden require()	MQTT dinleyici, alert üretici, WebSocket client
sumo_fiware_bridge.js	150	socket_listener içinden başlar	Route4Sim.xml izleyici, araç ID mapper
watchRouteSync.js	167	Ayrı proses olarak çalışır	Frontend→SUMO XML kopyalama

Dosya	Satır	Çalışma Şekli	Açıklama
db.js	110	Modül olarak import edilir	MariaDB pool + MongoDB bağlantısı
hardware.js	116	Manuel: node hardware.js	CSV'den MQTT simülasyonu (musoshi003)
musoshi001.js	40	Manuel: node musoshi001.js	Tek araç MQTT debug dinleyici
insertMockData.js	121	Manuel: node insertMockData.js	MongoDB test verisi üretici
reloadDb.js	-	Manuel	DB modül yeniden yükleme
auto_converter.py	-	Manuel: python auto_converter.py	H5→ONNX model dönüştürücü

MODELS (Veritabanı Modelleri)

Dosya	Veritabanı	Tablo/Koleksiyon	Açıklama
vehicle.js	**MariaDB**	**vehicle_tracking**	**Araç konum ve durum verisi**
vehicleFiware.js	**MariaDB**	**vehicle_tracking_fiware**	**FIWARE araç verileri**
customer.js	**MongoDB**	**customers**	**Müşteri bilgileri**

Dosya	Veritabanı	Tablo/Koleksiyon	Açıklama
performance.js	**MongoDB**	**anlikPerformansRota**	**Performans verileri**
PerformanceData2.js	**MongoDB**	**performanceData2**	**Genişletilmiş performans**
VehicleEnergy.js	**MongoDB**	**vehicleEnergy**	**Araç enerji tüketimi**
orderStatus.js	**MongoDB**	**orderStatus**	**Sipariş durumları**
chatbox.js	**MongoDB**	**chatbox**	**Sohbet mesajları**
User.js	**MongoDB**	**users**	**Kullanıcı hesapları**

ROUTES (API Endpoint'leri)

Dosya	Prefix	Metotlar	Açıklama
vehicles.js	/api/vehicles	GET, POST, PUT, DELETE	Araç CRUD, konum, durum, export
route.js	/api/routes	GET, POST	Rota yönetimi
customers.js	/api/customers	GET, POST, PUT, DELETE	Müşteri CRUD

Dosya	Prefix	Metotlar	Açıklama
orders.js	/api/orders	GET, POST, PUT	Sipariş yönetimi
performance.js	/api/performance	GET	Performans verileri
astar.js	/api/astar	POST	A* rota hesaplama
federated.js	/api/federated	GET, POST	Federated Learning kontrolü
auth.js	/api/auth	POST	Login, Register, Logout
chatbox.js	/api/chatbox	GET, POST	Sohbet mesajları
metrics.js	/api/metrics	GET	Uygulama metrikleri
prometheus.js	/metrics	GET	Prometheus scrape endpoint
reset.js	/api/reset	POST	Sistem reset işlemleri
route4vehicle.js	/api/route4vehicle	GET, POST	Araç-rota eşleştirme

MIDDLEWARE

Dosya	Açıklama
metrics.js	Prometheus metrik toplama middleware

UTILS

Dosya	Açıklama
parseRoute4Plan.js	XML rota dosyası ayrıştırıcı

musoshi_mock/ (Test Verileri)

Dosya	Araç	Açıklama
musoshi001-25LC.csv	musoshi001	25°C, Düşük Yük, Şehir içi
musoshi002-25LM.csv	musoshi002	25°C, Düşük Yük, Karma
musoshi003-25UC.csv	musoshi003	25°C, Yüksek Yük, Şehir içi
musoshi004-25UM.csv	musoshi004	25°C, Yüksek Yük, Karma
musoshi005-35LC.csv	musoshi005	35°C, Düşük Yük, Şehir içi
musoshi006-35UC.csv	musoshi006	35°C, Yüksek Yük, Şehir içi
musoshi007-35UM.csv	musoshi007	35°C, Yüksek Yük, Karma

FRONTEND KLASÖRÜ

Dosya	Açıklama
package.json	NPM bağımlılıkları

Dosya	Açıklama
.env.development	Geliştirme ortamı değişkenleri
.env.production	Üretim ortamı değişkenleri
README.md	Proje dokümantasyonu

src/ (Kaynak Kod)

Dosya	Açıklama
index.js	React giriş noktası
App.js	Ana uygulama bileşeni, routing
api.js	Backend API çağrıları
App.css	Global stiller
index.css	Kök stiller

src/pages/ (Sayfalar)

Dosya	Route	Açıklama
Home.js	/	Ana sayfa
Login.js	/login	Giriş sayfası
Register.js	/register	Kayıt sayfası
FleetMonitoring.js	/fleet	Filo izleme (3045 satır)

Dosya	Route	Açıklama
RouteOptimization.js	/routes	Rota optimizasyonu
RouteOptimizationSuit.js	/route-suit	Gelişmiş rota optimizasyonu
Vehicles.js	/vehicles	Araç yönetimi
Orders.js	/orders	Sipariş yönetimi
Performance.js	/performance	Performans dashboard
EVCharging.js	/charging	Şarj istasyonları
DemandPlanning.js	/demand	Talep planlama
Maintenance.js	/maintenance	Bakım yönetimi
Inventory.js	/inventory	Envanter
AdminPanel.js	/admin	Yönetici paneli
BpmnProcess.js	/bpmn	BPMN süreç görüntüleme

src/components/ (Bileşenler)

Dosya	Açıklama

Dosya	Açıklama
Navbar.js	Navigasyon çubuğu
AlertBar.js	Uyarı gösterim bileşeni
ChatBox.js	Sohbet bileşeni
OrderManagement.js	Sipariş yönetim bileşeni
ProtectedRoute.js	Korumalı route wrapper
RouteDetailPanel.js	Rota detay paneli

src/components/FleetMonitoring/

Dosya	Açıklama
Alt bileşenler	FleetMonitoring sayfası için modüler bileşenler

src/components/RouteOptimization/

Dosya	Açıklama
Alt bileşenler	Rota optimizasyonu için modüler bileşenler

src/contexts/ (React Context)

Dosya	Açıklama
AuthContext.js	Kimlik doğrulama state yönetimi

Dosya	Açıklama
HealthContext.js	Sistem sağlık durumu context
ScenarioContext.js	Senaryo yönetimi context

src/services/ (Servis Modülleri)

Dosya	Açıklama
api.js	HTTP istekleri wrapper
optimizationService.js	Optimizasyon API çağrıları
taskService.js	Görev yönetimi servisi
XMLService.js	XML işleme servisi

src/utils/ (Yardımcı Fonksiyonlar)

Dosya	Açıklama
batteryUtils.js	Batarya hesaplamaları
metrics.js	Metrik hesaplamaları
parseRoutexml.js	XML rota ayrıştırma
xmlGenerator.js	XML dosya üretimi
xmlGenerator_s.js	Alternatif XML üretici
xmlParser.js	XML ayrıştırma

Dosya	Açıklama
xmlService.js	XML servis fonksiyonları

src/data/ (Statik Veri)

Dosya	Açıklama
vehicles.js	Araç tanımları
customers.js	Müşteri tanımları
routes.js	Rota tanımları
orders.js	Sipariş tanımları
drivers.js	Sürücü tanımları
chargingStations.js	Şarj istasyonu tanımları
stops.js	Durak tanımları

public/ (Statik Dosyalar)

Klasör	Açıklama
esogu_dataset/	ESOGU veri seti (görev, araç, şarj istasyonu XML'leri)
esogu_dataset_s/	Alternatif veri seti
xml_dataset/	XML veri seti
output/RML/	Üretilen rota XML çıktıları

MACHINE KLASÖRÜ (ML Servisi)

Dosya	Port	Çalışma	Açıklama
app.py	5002	py app.py	Flask ML API sunucusu
federated_training.py	-	app.py içinden çağrılır	Federated Learning eğitim döngüsü
global_model.pt	-	Model dosyası	Eğitilmiş global PyTorch modeli
features.json	-	Konfigürasyon	Model feature tanımları
federated_status.txt	-	Durum dosyası	FL eğitim durumu
requirements.txt	-	Bağımlılıklar	Python paketleri

Client Dosyaları (Federated Learning)

Dosya	Araç	Açıklama
client_musoshi001.py	musoshi001	FL client - araç 1
client_musoshi002.py	musoshi002	FL client - araç 2
client_musoshi003.py	musoshi003	FL client - araç 3
client_musoshi004.py	musoshi004	FL client - araç 4

Dosya	Araç	Açıklama
client_musoshi005.py	musoshi005	FL client - araç 5
client_musoshi006.py	musoshi006	FL client - araç 6
client_musoshi007.py	musoshi007	FL client - araç 7
client_musoshi008.py	musoshi008	FL client - araç 8
client_musoshi009.py	musoshi009	FL client - araç 9
client_musoshi010.py	musoshi010	FL client - araç 10

app.py API Endpoint'leri

Endpoint	Metod	Açıklama
/predict	POST	Enerji tüketim tahmini
/federated/status	GET	FL eğitim durumu
/federated/start	POST	FL eğitim başlat
/federated/stop	POST	FL eğitim durdur
/model/info	GET	Model bilgileri

ROUTE-SERVICE KLASÖRÜ (Rota Servisi)

Dosya	Port	Açıklama
app.py	5555	Flask API giriş noktası

Dosya	Port	Açıklama
routes.py	-	API endpoint tanımları
routing.py	-	Rota hesaplama mantığı
graph_manager.py	-	OSMnx graf yönetimi
osmnx_astar_edge_based.py	-	Edge-based A* algoritması
geometry.py	-	Geometri hesaplamaları
database.py	-	Veritabanı bağlantısı
config.py	-	Konfigürasyon ayarları
test_service.py	-	Birim testleri
requirements.txt	-	Python bağımlılıkları

cache/ (Önbellek)

Dosya	Açıklama
esogu_drive.graphml	ESOGU kampüs yol ağı grafı
*.json	Önbelleğe alınmış rota sonuçları

API Endpoint'leri

Endpoint	Metod	Açıklama
/api/route	POST	A* rota hesapla
/api/route/depot	POST	Depoya rota hesapla
/api/graph/info	GET	Graf bilgileri
/health	GET	Servis sağlık kontrolü

SUMO KLASÖRÜ (Simülasyon)

Dosya	Port	Çalışma	Açıklama
main.py	8001	py main.py	Flask API + SUMO kontrolü
function.py	-	main.py içinden çağrılır	Simülasyon mantığı (844 satır)
ortools_func.py	-	İmport edilir	OR-Tools VRP çözücü
solver_and_printer.py	-	İmport edilir	Rota çözücü ve yazdırıcı
route_to_sumo.py	-	İmport edilir	Rota→SUMO dönüştürücü
database_create_routes.py	-	İmport edilir	DB'ye rota yazma
utils.py	-	İmport edilir	Yardımcı fonksiyonlar

Dosya	Port	Çalışma	Açıklama
filepath_constants.py	-	İmport edilir	Dosya yolu sabitleri

main.py API Endpoint'leri

Endpoint	Metod	Açıklama
sumo	POST	Simülasyon başlat
/sumo_reset	POST	Simülasyon sıfırla
/router	GET, POST	Rota hesapla (OR-Tools)
/writeroutedetails	GET, POST	SUMO rota dosyası oluştur
/writeroutedetails/<alg>	GET, POST	Belirli algoritma için rota
/api/vehicles/locations	GET	Araç konumları
/api/route4sim/check-load	GET, POST	Araç listesi
/api/astar_calculate_depot_bulk	GET, POST	Toplu A* hesaplama
/api/astar_apply_depot	POST	A* rotasını uygula

Endpoint	Metod	Açıklama
/api/vehicles/stop	POST	Aracı durdur
/api/vehicles/start_movement	POST	Aracı başlat
/api/resume_simulation	GET, POST	Simülasyonu devam ettir

algorithms/ (Algoritmalar)

Dosya	Açıklama
astar.py	A* en kısa yol algoritması
dijkstra.py	Dijkstra algoritması

ESOGU_SUMO/ (SUMO Dosyaları)

Dosya	Açıklama
osm.net.xml	ESOGU kampüs yol ağı
Route4Sim.xml	Simülasyon araç rotaları
cs.add.xml	Şarj istasyonu tanımları
*.sumocfg	SUMO konfigürasyon dosyaları

utils/ (Yardımcı Araçlar)

Dosya	Açıklama
db_connect.py	Veritabanı bağlantısı

Dosya	Açıklama
db_connect_order.py	Sipariş DB bağlantısı
create_random_orders.py	Rastgele sipariş üretici
xml_to_db.ipynb	XML→DB dönüşüm notebook

DEPLOY KLASÖRÜ (Kubernetes)

Dosya	Açıklama
namespaces.yaml	K8s namespace tanımları

monitoring/ (İzleme Konfigürasyonları)

Dosya Tipi	Dosyalar	Açıklama
Alert Rules	1-*.yaml	Prometheus alert kuralları
Exporters	2-*.yaml	Metrik exporter deployment'ları
Endpoints	3-*.yaml	Servis endpoint tanımları
Secrets	*-secret.yaml	Gizli bilgiler (credentials)
Blackbox	blackbox-*.yaml	Blackbox probe konfigürasyonları

Alert Dosyaları

Dosya	Açıklama

Dosya	Açıklama
1-backend-alerts.yaml	Backend servis alertleri
1-frontend-alerts.yaml	Frontend servis alertleri
1-mariadb-alerts.yaml	MariaDB alertleri
1-docker-alerts.yaml	Docker container alertleri
1-server-hardware-alerts.yaml	Sunucu donanım alertleri
fleet-alerts.yaml	Filo yönetimi özel alertleri
mongodb-atlas-alerts.yaml	MongoDB Atlas alertleri

Exporter Dosyaları

Dosya	Port	Açıklama
2-backend-exporter.yaml	9100	Backend metrik exporter
2-frontend-exporter.yaml	9100	Frontend metrik exporter
2-mariadb-exporter.yaml	9104	MariaDB metrik exporter
2-server-hardware-exporter.yaml	9101	Donanım metrik exporter
mongodb-exporter.yaml	9216	MongoDB metrik

Dosya	Port	Açıklama
		exporter

PORT ÖZETİ

Port	Servis	Teknoloji
3000	Frontend	React
3001	Backend API	Node.js/Express
5002	Machine Learning	Python/Flask
5555	Route Service	Python/Flask
8001	SUMO API	Python/Flask
8035	WebSocket Server	SUMO tarafı
1885	MQTT Broker	Mosquitto
3306	MariaDB	Database
27017	MongoDB Atlas	Database (Cloud)
9090	Prometheus	Monitoring

VERİTABANI BAĞLANTILARI

Veritabanı	Host	Port	Database	Kullanım
MariaDB	127.0.0.1	3306	fleetmanagementdb	Araç takip, FIWARE verisi

Veritabanı	Host	Port	Database	Kullanım
MongoDB Atlas	Cloud	-	RouteManagementDB	Alertler, müşteriler, performans

13. Backend Katmanı Detaylı İnceleme

13.1 Backend Klasör Yapısı

```
backend/
├── server.js              # Ana uygulama giriş noktası
├── db.js                  # Veritabanı bağlantı yönetimi
├── socket_listener.js     # MQTT ve WebSocket dinleyici
├── sumo_fiware_bridge.js  # SUMO-FIWARE köprüsü
├── watchRouteSync.js      # Rota dosyası izleyici
├── hardware.js            # CSV simülatör
├── reloadDb.js            # DB modül yeniden yükleme
├── insertMockData.js      # Test verisi üretici
├── musoshi001.js          # Tekil araç MQTT dinleyici
├── auto_converter.py      # Model dönüştürücü (H5→ONNX)
├── middleware/
│   └── metrics.js         # Prometheus metrik toplama
├── models/
│   ├── vehicle.js         # Araç veri modeli (MariaDB)
│   ├── vehicleFiware.js   # FIWARE araç modeli
│   ├── customer.js        # Müşteri modeli (MongoDB)
│   ├── performance.js     # Performans verisi modeli
│   ├── PerformanceData2.js  # Genişletilmiş performans modeli
│   ├── VehicleEnergy.js   # Araç enerji modeli
```

```
│   ├── orderStatus.js       # Sipariş durumu modeli
│   ├── chatbox.js           # Sohbet modeli
│   └── User.js              # Kullanıcı modeli
├── routes/
│   ├── vehicles.js          # Araç API endpoint'leri
│   ├── route.js             # Rota API endpoint'leri
│   ├── customers.js         # Müşteri API endpoint'leri
│   ├── orders.js            # Sipariş API endpoint'leri
│   ├── performance.js       # Performans API endpoint'leri
│   ├── astar.js             # A* rota API endpoint'leri
│   ├── federated.js         # Federated Learning API
│   ├── auth.js              # Kimlik doğrulama
│   ├── chatbox.js           # Sohbet API
│   ├── metrics.js           # Metrik API
│   ├── prometheus.js        # Prometheus endpoint
│   ├── reset.js             # Reset işlemleri
│   └── route4vehicle.js     # Araç-rota eşleştirme
├── utils/
│   └── parseRoute4Plan.js   # XML rota ayrıştırıcı
└── musoshi_mock/
    └── *.csv                # Mock araç verileri
```

13.2 server.js (Ana Uygulama Giriş Noktası)

`server.js` dosyası 4074 satır kod içeren, sistemin merkezi koordinasyon noktasıdır. Aşağıdaki mantıksal bölümlerle yapılandırılmıştır:

13.2.1 Modül İçe Aktarma ve Yapılandırma (Satır 1-50)

Yüklenen Temel Modüller:

- express: HTTP sunucu çerçevesi

- cors: Cross-Origin Resource Sharing

- mongoose: MongoDB ODM
- multer: Dosya yükleme
- xml2js: XML ayrıştırma/oluşturma
- dotenv: Ortam değişkenleri

Uygulama başlangıcında tüm rota işleyicileri yüklenir ve doğrulanır:

```javascript
const vehicleRoutes = require('./routes/vehicles');
const routeRoutes = require('./routes/route');
const customerRoutes = require('./routes/customers');
const performanceRoutes = require('./routes/performance');
const chatboxRoutes = require('./routes/chatbox');
const federatedRoutes = require('./routes/federated');
const ordersRouter = require('./routes/orders');
const astarRoutes = require('./routes/astar');
```

13.2.2 Kubernetes Bootstrap Entegrasyonu (Satır 32-60)

Sistem, Kubernetes ortamında çalıştığında otomatik olarak izleme altyapısını başlatır:

- `AUTO_BOOTSTRAP_K8S` ortam değişkeni kontrol edilir
- Prometheus erişilebilirliği test edilir
- Gerekirse arka planda bootstrap script'i çalıştırılır

13.2.3 Otomatik Port-Forward Mekanizması (Satır 62-120)

Kubernetes servisleri için otomatik port yönlendirme sağlar:

Prometheus | 9090 | 9090 | monitoring

Fleet Health | 8500 | 8080 | fleet

```javascript
// Port kullanım kontrolü
function portInUse(port) {
  return new Promise((resolve) => {
    const srv = net.createServer()
```

```
        .once("error", () => resolve(true))
        .once("listening", () => srv.close(() => resolve(false)))
        .listen(port, "127.0.0.1");
    });
}
```

13.2.4 CORS ve Middleware Yapılandırması (Satır 140-200)

```javascript
app.use(cors({
    origin: function (origin, callback) {
        const allowedOrigins = [
            'http://localhost:3000',
            'http://127.0.0.1:3000'
        ];
        // Geliştirme ortamında tüm originlere izin ver
        callback(null, true);
    },
    credentials: true,
    methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH', 'OPTIONS'],
    allowedHeaders: ['Content-Type', 'Authorization', 'Accept']
}));
```

13.2.5 Veritabanı Bağlantıları (Satır 250-290)

İki ayrı MongoDB bağlantısı yönetilir:

1. **Ana Veritabanı** (`MONGODB_URI`): RouteManagementDB
2. **Musoshi Veritabanı** (`MONGODB_MUSOSHI`): OPEVA araç rota veritabanı

```javascript
// Ana MongoDB bağlantısı
```

```
mongoose.connect(MONGODB_URI, {
    useNewUrlParser: true,
    useUnifiedTopology: true,
});
```

(musoshi veri tabanı kullanımdan kaldırılmıştır ve system bağımsız hale getirilmiştir.)

```
// Musoshi MongoDB bağlantısı (rota gönderimi için)
const connectToMusoshiMongoDB = async () => {
    musoshiMongoClient = new MongoClient(musoshiUri);
    await musoshiMongoClient.connect();
    musoshiDb = musoshiMongoClient.db('opeva');
};
```

13.2.6 Socket ve FIWARE Entegrasyonu (Satır 140)

WebSocket dinleyicisi ana sunucu başlatılmadan önce aktive edilir:

```javascript
require('./socket_listener'); // WebSocket entegrasyonu
```

Bu satır, MQTT broker'a bağlanmayı ve gerçek zamanlı araç verisi almayı başlatır.

13.2.7 Route Tanımları ve API Endpoint'leri (Satır 300-4000+)

Ana API rotaları şu şekilde kaydedilir:

```javascript
app.use('/api/vehicles', vehicleRoutes);
app.use('/api/routes', routeRoutes);
app.use('/api/customers', customerRoutes);
app.use('/api/performance', performanceRoutes);
app.use('/api', ordersRouter);
app.use('/api/astar', astarRoutes);
```

app.use('/api/federated', federatedRoutes);
```

### Önemli Inline Endpoint'ler

/api/test-delivery` | GET | API sağlık kontrolü

/api/debug-routes` | GET | Kayıtlı rotaları listele

/health/summary` | GET | Fleet-health proxy

/rml/update-xml-info4algorithm` | POST | Algoritma parametrelerini güncelle |

/api/customers/import-from-xml` | POST | XML'den müşteri içe aktar |

/rml/routes/route4simparser` | POST | SUMO rota dosyası oluştur |

/api/savepickupdelivery` | GET/POST | Teslimat verisi önbelleği |

/api/orders/get-waiting-orders` | GET | Bekleyen siparişleri getir |

### 13.2.8 Arka Plan İzleyiciler

Sunucu başlatıldığında aktive edilen izleyiciler:

1. **socket_listener.js**: MQTT mesajlarını dinler
2. **sumo_fiware_bridge.js**: Route4Sim.xml değişikliklerini izler
3. **watchRouteSync.js**: RML çıktı dosyalarını SUMO'ya kopyalar

### 13.2.9 Port ve Çalışma Davranışı

```javascript
const port = process.env.PORT || 3001;
app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

13.3 Backend Yardımcı Dosyalar (Dosya Bazlı Liste)

Dosya Adı: db.jsAmacı: Veritabanı bağlantı havuzu yönetimi ve şema tanımlamalarını içerir. Hem MariaDB hem de MongoDB bağlantılarını tek bir modülde birleştirir.

Temel İşlevler:

MariaDB connection pool oluşturma ve yönetme
```

MongoDB Mongoose bağlantısı

Alert şeması tanımlama ve indeksleme

Yapılandırma Parametreleri:

```javascript
const poolConfig = {
    connectionLimit: 20,      // Havuz kapasitesi
    acquireTimeout: 20000,    // 20 saniye bekleme
    idleTimeout: 60000,       // 60sn boş bağlantı timeout
    initSql: 'SET SESSION wait_timeout=60'
};
```

Bağımlı Modüller:

socket_listener.js`: Araç verisi kaydetme

`routes/vehicles.js`: Araç sorguları

`routes/federated.js`: ML veri çekme

Tüm model dosyaları

Dışa Aktarılan Öğeler:

```javascript
module.exports = { Alert, pool, getConnection, connectDB };
```

Dosya Adı: socket_listener.js

Amacı:

MQTT broker ve WebSocket sunucusundan gerçek zamanlı araç verisi alır, veritabanına kaydeder ve otomatik uyarı üretir.

Dinlenen Kaynaklar:

MQTT Broker | localhost:1885 | Araç telemetrisi

WebSocket | 0.0.0.0:8035 | Rota ve simülasyon verisi

Dinlenen Araçlar:

```javascript
const REAL_VEHICLES = ["musoshi001", "abc", "musoshi003", "musoshi006"];
const SUMO_VEHICLES = ["musoshi004", "musoshi005", "musoshi007",
                       "musoshi008", "musoshi009", "musoshi010"];
```

MQTT Topic Formatı

```
/json/secret/{vehicle_id}/attrs
```

Otomatik Uyarı Sistemi:

Sistem, gelen verileri analiz ederek aşağıdaki kategorilerde uyarı üretir:

BATTERY | <20% Critical, <30% Warning, <40% Info | Error/Warning/Info

VOLTAGE | <47V Critical, <48.5V Warning | Error/Warning

LOAD | >1200kg Critical, >800kg Warning | Error/Warning

THERMAL | >30°C Critical, >25°C Warning | Error/Warning

SPEED | >50 km/h Critical, >30 km/h Warning | Error/Warning

RANGE | <30km Critical, <50km Warning | Error/Warning

Veri Kayıt Akışı:

1. MQTT/WebSocket'ten veri alınır
2. Veri doğrulanır (latitude/longitude kontrolü)
3. `insertVehicleData()` ile MariaDB'ye yazılır
4. `generateDynamicAlerts()` ile uyarılar üretilir
5. MongoDB'ye uyarılar kaydedilir

Dosya Adı: sumo_fiware_bridge.js

Amacı: SUMO simülasyonunda kullanılan Route4Sim.xml dosyasını izler ve araç ID eşleştirmesini yönetir.

İzlenen Dosya Konumları:

```javascript
```

```
const POSSIBLE_ROUTE_PATHS = [
  '../Sumo/ESOGU_SUMO/Route4Sim.xml',
  'C:/Users/S/Desktop/Application/app/app/Sumo/ESOGU_SUMO/Route4Sim.xml'
];
```

Araç ID Eşleştirmesi:

SUMO'da veh0, veh1 gibi isimlendirilen araçlar FIWARE sisteminde musoshi004, musoshi005 gibi isimlerle yayınlanır.

```javascript
const FIWARE_VEHICLES = ["musoshi004", "musoshi005", "musoshi007",
                         "musoshi008", "musoshi009", "musoshi010"];
```

Çalışma Mekanizması:

1. `chokidar` kütüphanesi ile dosya değişiklikleri izlenir
2. XML parse edilerek araç listesi çıkarılır
3. Eşleştirme JSON dosyasına yazılır: `vehicle_mapping.json`
4. Python script'leri bu dosyayı okuyarak doğru FIWARE topic'lerine yayın yapar

Dosya Adı: watchRouteSync.js

Amacı:

Frontend'in ürettiği rota dosyalarını (RML çıktısı) SUMO simülasyon dizinine otomatik olarak kopyalar ve araç ID'lerini dönüştürür.

İzlenen Dizin:

```
frontend/public/output/RML/
```

Dosya Eşleştirmeleri:

```javascript
const FILE_MAPPINGS = [
  {
    name: 'Task_Route4Sim.xml',
```

```
        src: watchedDir + '/Task_Route4Sim.xml',
        dest: 'Sumo/ESOGU_SUMO/Route4Sim.xml'
    },
    {
        name: 'Test1_Route4Sim.xml',
        src: watchedDir + '/Test1_Route4Sim.xml',
        dest: 'Sumo/ESOGU_SUMO/Route4Sim.xml'
    }
];
```

Araç ID Dönüştürme:

```javascript
const VEHICLE_MAPPING = {
    'veh0': 'musoshi004',
    'veh1': 'musoshi005',
    'veh2': 'musoshi007',
    'veh3': 'musoshi008',
    'veh4': 'musoshi009',
    'veh5': 'musoshi010'
};
```

Dosya kopyalanırken `id="veh0"` ifadeleri `id="musoshi004"` olarak değiştirilir.

Dosya Adı: hardware.js

Amacı:

CSV dosyasından okunan araç verilerini MQTT broker'a yayınlayarak gerçek araç simülasyonu yapar.

Yapılandırma:

```javascript
const VEHICLE_ID = "musoshi003";
```

```
const csvFilePath = "./vehicleData_t_0_table.csv";
const topic = `/json/secret/${VEHICLE_ID}/attrs`;
```

Veri Dönüşümü:

CSV satırları, FIWARE formatına dönüştürülerek yayınlanır:

Konum bilgisi (latitude, longitude, altitude)

Batarya durumu (SoC, PACK_V_SUM_OF_CELLS, PACK_I_HALL)

Hücre sıcaklıkları (CELL_T_MIN_VAL, CELL_T_MAX_VAL)

Motor verileri (DC_Link_Voltage, MotorTemperature)

Yayın Döngüsü:

CSV sonuna ulaşıldığında yeni bir `tourID` oluşturularak başa dönülür.

Dosya Adı: reloadDb.js

Amacı:

Geliştirme sırasında db.js modülünün cache'ini temizleyerek yeniden yüklenmesini sağlar.

Kullanım Senaryosu

Hot-reload durumlarında veritabanı bağlantısının tutarlılığını korumak için kullanılır.

Dosya Adı: insertMockData.js

Amacı:

Test ve geliştirme amaçlı olarak MongoDB'ye örnek performans verisi üretir.

Üretilen Veriler:

3 farklı rota için simüle edilmiş veriler

Her 5 saniyede güncellenen değerler

Değişken hız, konum ve ağırlık değerleri

```javascript
// Örnek veri yapısı
{
  route_id: "route_001",
```

```
  vehicle_id: "vehicle_A1",
  timestamp: new Date(),
  location: { lat: 38.45, lon: 27.15 },
  avg_vehicle_speed: 40.5,
  avg_Total_Mass: 2500.0,
  status: "moving"
}
```

Dosya Adı: musoshi001.js

Amacı:

Tekil bir araç (musoshi001) için MQTT dinleyici örneği. Diğer araçlar için şablon görevi görür.

Çalışma Şekli:

```javascript
const TOPIC = `/json/secret/musoshi001/attrs`;
client.subscribe(TOPIC, { qos: 1 });
```

Gelen veriler konsola loglanır ve isteğe bağlı olarak başka bir broker'a yönlendirilebilir.

Dosya Adı: auto_converter.py

Amacı

Keras/TensorFlow H5 formatındaki ML modellerini ONNX formatına dönüştürür.

Çözülen Problemler:

1. H5 dosyalarındaki encoding sorunları
2. Model yapılandırma UTF-8 hataları
3. TensorFlow-ONNX dönüşüm uyumsuzlukları

Dönüşüm Adımları:

1. H5 dosyasındaki encoding hataları düzeltilir
2. TensorFlow modeli yüklenir

3. tf2onnx ile ONNX formatına dönüştürülür

4. Çıktı dosyası kaydedilir

13.4 Middleware Detayları

middleware/metrics.js

Amacı:

Prometheus formatında metrik toplama ve dışa aktarma işlemlerini yönetir.

Toplanan Metrikler:

http_request_duration_seconds` | Histogram | HTTP istek süreleri

http_requests_total` | Counter | Toplam HTTP istek sayısı

http_response_size_bytes` | Summary | Yanıt boyutları

nodejs_*` | Default | CPU, bellek, GC metrikleri

Endpoint:

```
GET /metrics → Prometheus scrape endpoint
```

13.5 Model Dosyaları

models/vehicle.js (MariaDB)

Amacı:

MariaDB'deki `vehicle_tracking` tablosundan en güncel araç konumlarını sorgular.

SQL Sorgusu:

```sql
SELECT vt.vehicle_id, vt.latitude, vt.longitude, vt.speed,
       vt.state_of_charge, vt.planned_path, vt.actual_path,
       vt.delivery_stops, vt.charging_stops
FROM vehicle_tracking vt
INNER JOIN (
   SELECT vehicle_id, MAX(last_updated) AS max_updated
```

```
    FROM vehicle_tracking GROUP BY vehicle_id
) latest ON vt.vehicle_id = latest.vehicle_id
```

JSON Alanları Parse Edilir:

planned_path

actual_path

delivery_stops

charging_stops

models/customer.js (MongoDB)

Amacı:

Müşteri bilgilerini MongoDB'de yönetir.

Şema:

```javascript
{
    name: String,
    address: String,
    demand: Number,
    timeWindow: String,
    orderDate: String,
    status: ['Requested', 'On the way', 'Delivered', 'Cancelled'],
    brand: String,
    notes: String
}
```

models/performance.js (MongoDB)

Amacı:

Rota performans metriklerini saklar.

Şema Alanları:

`route_id`: Rota tanımlayıcısı

`avg_energy_consumption_kwh_km`: Ortalama enerji tüketimi

`estimated_total_energy_kwh`: Tahmini toplam enerji

`total_distance_km`: Toplam mesafe

`efficiency_score`: Verimlilik puanı

13.6 Route Dosyaları (API Endpoint'leri)

routes/vehicles.js

Sunulan Endpoint'ler:

`/api/vehicles/locations` | GET | Tüm araç konumlarını getir

`/api/vehicles/locations/fiware` | GET | FIWARE araç verilerini getir

`/api/vehicles/export` | GET | Araç verilerini dışa aktarır.

routes/route.js

Amacı:

MongoDB'deki rota verilerine erişim sağlar.

Endpoint'ler:

`/api/routes` | GET | Tüm rotaları listele

`/api/routes/:routeId` | GET | Belirli rotayı getir

routes/orders.js

Amacı:

SUMO ve FIWARE sipariş istatistiklerini birleştirir.

Dönen Veri Yapısı:

```javascript
{
  sumoStats: {
    totalVehicles: number,
    activeDeliveries: number,
```

```
        completedDeliveries: number
    },
    fiwareStats: {
        totalVehicles: number,
        activeRoutes: number,
        completedDeliveries: number
    },
    orders: []
}
```

routes/astar.js

Amacı:

A* algoritması ile hesaplanan rotaları Route4Vehicle.json dosyasına ekler.

Ana Endpoint:

```
POST /api/astar/add-route
```

İstek Gövdesi:

```javascript
{
    vehicleId: string,
    astarRoute: array,      // Edge listesi
    routeId: string,
    currentVehiclePosition: object,
    destinationType: 'first_customer' | 'depot'
}
```

routes/federated.js

Amacı:

Federated Learning için araç verilerini çeker.

Endpoint:

```
GET /api/vehicle-data/musoshi003/:route_tour_id
```

Dönen Veri:

```javascript
[
  { soc, PACK_V_SUM_OF_CELLS, PACK_I_HALL, CELL_T_MAX_VAL }
]
```

13.7 Utility Dosyaları

utils/parseRoute4Plan.js

Amacı:

Route4Plan.xml dosyasını parse ederek müşteri bilgilerini MongoDB'ye kaydeder.

İşlev:

```javascript
async function parseAndSaveCustomersFromXML(xmlFilePath) {
  // XML'den Solution/Routes/Route/Nodes/Node bilgisini çıkar
  // NodeType === 'Customer' olanları veritabanına kaydet
}
```

14. Frontend Katmanı Genel Mantığı

14.1 Frontend-Backend İletişimi

Frontend, backend ile REST API üzerinden haberleşir. Tüm API çağrıları `services/api.js` dosyasında merkezi olarak tanımlanmıştır.

API Endpoint Tanımları:

```javascript
const API_BASE_URL = 'http://localhost:3001/api';
const EXTERNAL_PREDICT_URL = 'http://localhost:5002';
const ALNS_API_URL = 'http://localhost:8005';

export const ENDPOINTS = {
  ALERTS: `${API_BASE_URL}/alerts`,
  PERFORMANCE: `${API_BASE_URL}/performance`,
  VEHICLES: `${API_BASE_URL}/sumo/vehicles/all`,
  ENERGY_PREDICT: `${EXTERNAL_PREDICT_URL}/predict`,
  SOC_PREDICT: `${EXTERNAL_PREDICT_URL}/estimate-soc`,
  FEDERATED_SOC: `${EXTERNAL_PREDICT_URL}/federated-soc`,
  START_ALNS: `${ALNS_API_URL}/start_alns`,
};
```

14.2 Uygulama Yapısı (App.js)

Ana uygulama bileşeni aşağıdaki sorumlulukları üstlenir:

1. **Kimlik Doğrulama**: AuthContext ile kullanıcı oturumu yönetimi
2. **Yetkilendirme**: ProtectedRoute ile rol bazlı erişim kontrolü
3. **Tema Yönetimi**: Dark/Light mode desteği
4. **Health Monitoring**: HealthProvider ile sistem durumu izleme

Sayfa Rotaları ve Yetkilendirme:

Home | / | Tümü

Route Optimization | /route-optimization | admin, driver

Fleet Monitoring | /fleet-monitoring | admin, driver, customer

Maintenance | /maintenance | admin

EV Charging | /ev-charging | admin

Performance | /performance-monitoring | admin

Vehicles | /vehicles | admin

Orders | /orders | admin

Admin Panel | /admin-panel | admin

14.3 Fleet Monitoring Ekranı

Fleet Monitoring (`pages/FleetMonitoring.js`) uygulamanın en karmaşık ekranıdır (3045+ satır).

Ana İşlevler:

1. Gerçek Zamanlı Araç Takibi
   - FIWARE ve SUMO araçlarının harita üzerinde gösterimi
   - Araç yönü (bearing) hesaplama
   - Planlanan ve gerçekleşen rota çizimi
2. Uyarı Yönetimi
   - Severity bazlı filtreleme
   - Uyarı çözme işlemi
   - Gerçek zamanlı güncelleme
3. Performans İzleme
   - Enerji tüketim grafikleri
   - SoC (State of Charge) tahminleri
   - Rota verimlilik skorları
4. A* Rota Hesaplama
   - Depot'a dönüş rotası

İlk müşteriye rota

Dinamik rota güncelleme

Araç ID Eşleştirmeleri:

```javascript
const vehicleIdMap = {
  veh0: "musoshi004",
  veh1: "musoshi005",
  veh2: "musoshi007",
  veh3: "musoshi008",
  veh4: "musoshi009",
  veh5: "musoshi010"
};
```

14.4 Route Optimization Ekranı

Rota optimizasyon ekranı (`pages/RouteOptimization.js`) basit bir yapıya sahiptir ve asıl karmaşıklık `components/RouteOptimization/Layout.js` bileşenindedir.

Desteklenen Algoritmalar:

Simulated Annealing

Tabu Search

ALNS (Adaptive Large Neighborhood Search)

DQN

DDQN

Q-LEARNING

Veri Kaynakları:

`data/vehicles.js`: Statik araç verileri

`data/customers.js`: Müşteri bilgileri

`data/chargingStations.js`: Şarj istasyonları

`data/routes.js`: Örnek rotalar

14.5 Kimlik Doğrulama Sistemi

`contexts/AuthContext.js` dosyası kimlik doğrulama mantığını yönetir.

Sağlanan İşlevler:

```javascript
{
  login: async (credentials) => {},
  logout: async () => {},
  register: async (userData) => {},
  getAllUsers: async () => {},  // Admin only
  updateUser: async (userId, userData) => {},
  deleteUser: async (userId) => {}
}
```

Oturum Yönetimi:

Session ID localStorage'da saklanır

Her istekte session doğrulaması yapılır

15. Makine Öğrenmesi ve Tahmin Servisleri

15.1 Machine Klasörü Yapısı

```
machine/
├── app.py                  # Flask API sunucusu
├── federated_training.py   # Federated Learning eğitimi
├── client_musoshi001.py    # FL istemci örneği
├── client_musoshi002.py    # ... (010'a kadar)
├── best_model.pkl          # Eğitilmiş CatBoost modeli
├── global_model.pt         # Federated LSTM modeli
```

```
├── scaler_x.pkl            # Girdi normalizer
├── scaler_y.pkl            # Çıktı normalizer
├── features.json           # Model özellikleri listesi
├── requirements.txt        # Python bağımlılıkları
└── data/
    └── data.csv            # Eğitim verisi
```

15.2 app.py - Ana Tahmin Servisi

Port: 5002

Sunulan Endpoint'ler:

`/predict` (POST)

Amacı: Segment bazlı enerji tüketimi tahmini

Girdi Özellikleri:

```javascript
[
    "segment_length",      // Segment uzunluğu (metre)
    "slope",               // Eğim (derece)
    "avg_vehicle_speed",   // Ortalama hız (km/h)
    "avg_Acceleration",    // Ortalama ivme (m/s²)
    "avg_Total_Mass"       // Toplam kütle (kg)
]
```

Çıktı:

```javascript
{
    "prediction": 45.2,            // Segment tüketimi (Wh)
    "shap_values": [...],          // SHAP açıklamaları
```

```
    "avg_consumption_wh_per_km": 116.3,
    "remaining_range": 42.5,       // Kalan menzil (km)
    "remaining_energy": 4920.0,    // Kalan enerji (Wh)
    "nonlinear_energy": 116.3      // Doğrusal olmayan enerji tüketimi
}
```

Model: CatBoost Regressor (`best_model.pkl`)

`/estimate-soc` (POST)

Amacı: LSTM modeli ile SoC (State of Charge) tahmini

Model Mimarisi:

```python
class LSTMModel(torch.nn.Module):
    def __init__(self, input_size=4, hidden_size=64, num_layers=2):
        self.lstm = torch.nn.LSTM(input_size, hidden_size, num_layers)
        self.fc = torch.nn.Linear(hidden_size, 1)
```

Girdi:

```javascript
{
    "vehicle_id": "musoshi003"
}
```

Veri Akışı:

1. Backend API'den son 10 kayıt çekilir
2. Özellikler normalize edilir (scaler_x)
3. LSTM modeli ile tahmin yapılır
4. Çıktı ters normalize edilir (scaler_y)

`/federated-soc` (POST)

Amacı: Federated Learning eğitimini başlatır

Süreç:

1. 7 araç için veri toplanır
2. Global scaler'lar fit edilir
3. Flower sunucusu başlatılır
4. İstemciler eğitilir
5. Model ağırlıkları birleştirilir
6. `global_model.pt` güncellenir

15.3 Federated Learning Mimarisi

Flower Framework Kullanımı:

```python
# Sunucu yapılandırması
strategy = SaveModelStrategy(
    fraction_fit=1.0,
    fraction_evaluate=0.5,
    min_fit_clients=3,
    min_evaluate_clients=3,
    min_available_clients=3

Sunucu başlatma
start_server(
    server_address="127.0.0.1:8080",
    config=ServerConfig(num_rounds=3),
    strategy=strategy
)
```

İstemci Yapısı:

Her araç için ayrı bir istemci scripti mevcuttur (`client_musoshi001.py` - `client_musoshi010.py`).

```python
class FlowerClient(NumPyClient):
    def fit(self, parameters, config):
        # Modeli güncelle
        set_parameters(self.model, parameters)
        # Yerel eğitim
        self.train(epochs=1)
        # Güncel parametreleri döndür
        return get_parameters(self.model), len(self.trainloader)
```

15.4 Tahmin Sonuçlarının Kullanımı

Tahmin servisleri Fleet Monitoring ekranında aşağıdaki şekillerde kullanılır:

1. **Menzil Tahmini**: Kalan batarya ile gidilebilecek mesafe
2. **Enerji Tüketim Analizi**: SHAP değerleri ile hangi faktörün etkili olduğu
3. **SoC Tahmini**: Batarya durumunun gelecek tahminleri
4. **Rota Optimizasyonu**: Enerji verimli rota seçimi

16. Rota ve Simülasyon Servisleri

16.1 Route-Service Yapısı

```
route-service/
├── app.py              # Flask API sunucusu
├── config.py           # Yapılandırma sabitleri
├── database.py         # MariaDB bağlantısı
```

```
├── geometry.py              # Geometrik hesaplamalar
├── graph_manager.py         # Graf yükleme ve önbellekleme
├── routing.py               # A* rota hesaplama
├── osmnx_astar_edge_based.py # Edge tabanlı A* implementasyonu
├── routes.py                # API endpoint'leri
└── cache/
    └── esogu_drive.graphml   # Önbelleklenmiş graf
```

16.2 app.py – Rota Servisi

Port: 5555

Sunulan Endpoint'ler:

Endpoint | Metod | İşlev

`/health` | GET | Servis sağlık kontrolü

`/preload-graph` | POST | Grafı önbelleğe yükle

`/api/fiware/vehicles/unique-list` | GET | FIWARE araç listesi

`/api/fiware/calculate-depot-route` | GET/POST | Depot rotası hesapla

`/calculate-depot-route` | POST | Genel rota hesaplama

16.3 Graf Yönetimi (graph_manager.py)

OSMnx Entegrasyonu:

Graf, OpenStreetMap verilerinden oluşturulur veya cache'den yüklenir

```python
GRAPHML_PATH = "cache/esogu_drive.graphml"
def get_or_create_net():
    if os.path.exists(GRAPHML_PATH):
        G = ox.load_graphml(GRAPHML_PATH)
    else:
```

```
        G = ox.graph_from_point(
            (DEPOT_LAT, DEPOT_LON),
            dist=POINT_DIST_M,
            network_type='drive'
        )
        ox.save_graphml(G, GRAPHML_PATH)
    return G
```

SUMO Net Desteği:

Alternatif olarak SUMO net dosyasından da graf oluşturulabilir.

16.4 Edge Tabanlı A* (osmnx_astar_edge_based.py)

Özellikler:

Her kenar (edge) ayrı bir durum olarak değerlendirilir

Haversine mesafe heuristiği kullanılır

Dinamik sanal düğüm desteği

Heuristik Fonksiyon:

```python
def heuristic(self, edge_a: EdgeId, edge_b: EdgeId) -> float:
    ca = self._get_edge_center(edge_a)
    cb = self._get_edge_center(edge_b)
    dist_km = self._haversine(ca[1], ca[0], cb[1], cb[0])
    return (dist_km / self.vmax_global) * 3600.0  # seconds
```

16.5 SUMO Simülasyon Servisi

```
Sumo/
├── main.py                 # Flask API + TraCI kontrolü
```

```
├── function.py              # Simülasyon fonksiyonları
├── route_to_sumo.py         # Rota→SUMO dönüşümü
├── ortools_func.py          # OR-Tools optimizasyon
├── solver_and_printer.py    # Çözüm yazdırma
├── database_create_routes.py # DB rota oluşturma
├── algorithms/
│   ├── astar.py             # SUMO A* implementasyonu
│   └── dijkstra.py          # Dijkstra implementasyonu
└── ESOGU_SUMO/
    ├── osm.net.xml          # SUMO network dosyası
    ├── Route4Sim.xml        # Simülasyon rotaları
    └── *.sumocfg            # SUMO yapılandırmaları
```

16.6 main.py - SUMO API Sunucusu

Port: 8001 **Sunulan Endpoint'ler:**

`/router` | GET/POST | Rota çözümü (OR-Tools)

`/writeroutedetails/<algorithm>` | GET/POST | SUMO rota dosyası oluştur

`/sumo` | POST | Simülasyonu başlat

`/sumo_reset` | POST | Simülasyonu sıfırla

`/api/vehicles/locations` | GET | Araç konumları

`/api/route4sim/check-load` | GET/POST | Route4Sim araç listesi

`/api/vehicle/stop` | POST | Aracı durdur

`/api/vehicle/resume` | POST | Aracı devam ettir

16.7 function.py - Simülasyon Mantığı

TraCI Entegrasyonu:

SUMO simülasyonu TraCI (Traffic Control Interface) üzerinden kontrol edilir.

FIWARE Veri Yayını:

Simülasyon verileri ayrı bir thread'de FIWARE broker'a yayınlanır:

```python
FIWARE_BROKER = "0.0.0.0"
FIWARE_PORT = 18883
def fiware_worker():
    client = mqtt.Client()
    client.connect(FIWARE_BROKER, FIWARE_PORT, 60)

    vehicle_mapping = {
        "veh0": "musoshi004",
        "veh1": "musoshi005",
        ...
    }
```

16.8 OR-Tools Optimizasyon (ortools_func.py)

Kullanılan Kısıtlar:

Kapasite kısıtı (200 kg)

Batarya kısıtı (9000 Wh)

Zaman penceresi

Maksimum mesafe

Veri Modeli:

```python
data['num_vehicles'] = 5
data['vehicle_capacity'] = 200
data['battery_capacity'] = 9000
data['vehicle_max_time'] = 10500
data['vehicle_speed'] = 5 * 60 / 3.6  # 5 km/h → m/min
```

```

## 16.9 Route4Sim.xml Formatı

SUMO simülasyonu için kullanılan XML formatı:

```xml
<routes>
  <vehicle id="musoshi004" type="evehicle" depart="0">
    <param key="actualBatteryCapacity" value="9000"/>
    <route edges="edge1 edge2 edge3"/>
    <stop chargingStation="cs1" duration="300"/>
    <stop containerStop="dp1" duration="120"/>
  </vehicle>
</routes>
```

17. Sağlık İzleme Servisleri

17.1 Sistem Genel Bakış

Bu sistem, profesyonel SRE standartlarında bir sağlık izleme (health monitoring) çözümüdür:

- 6 kritik servisi izler (Backend, Frontend, MariaDB, MQTT, Docker,Fleet/Network).
- Tüm sistemlerde toplam 75 alarmı (alerts) takip eder.
- Prometheus cardinality maliyetlerini minimize etmek için iki aşamalı kazıma (two-tier scraping: always-on ve debug) kullanır.
- Olay (incident) anında debug metrics'leri otomatik olarak etkinleştirir ve sorun çözüldüğünde devre dışı bırakır.
- Fleet-wide health aggregation ile genel sağlık durumunu (yeşil, sarı, kırmızı) raporlar.
- Akıllı alarm yönlendirme ve engelleme için Alertmanager ile entegre çalışır.
- Konteynerize Yapı: Tüm sistemin tek bir komutla ayağa kalkmasını sağlayan yapı sayesinde karmaşık orkestrasyon araçlarına ihtiyaç duyulmaz.
```

- Canlı Uyarı Çubuğu: Kullanıcı arayüzünde yer alan ve kritik bildirimleri anlık olarak ileten etkileşimli uyarı alanı.
- Admin Paneli: İzleme göstergeleri, alarm yönetimi araçları ve geçmiş kayıtların incelenebildiği kontrol merkezi.

Feature	Description
Docker Compose	Entire stack runs as containers, no Kubernetes required|
Dynamic Debug Scraping	Expensive metrics only scraped during incidents (automatic)
Hysteresis Control	10-minute cooldown prevents metric flapping
TTL Protection	Max 30-minute debug window prevents runaway costs
Alert Inhibition	Critical alerts suppress related warnings (reduces noise)
Fleet Health Dashboard	Real-time aggregated status across all services
Blackbox Probing	External TCP/ICMP connectivity monitoring
Algorithm Health Checks	8 routing algorithms monitored via blackbox probes
Container Control	Stop debug-controller container = all metrics flow

17.2 Mimari ve Tasarım

- Prometheus: Veri toplama merkezidir. Belirlenen hedefleri (exporters) her 60 saniyede bir tarayarak metrikleri kaydeder.
- Alertmanager: Prometheus'tan gelen uyarıları alır, gruplandırır ve webhook üzerinden Fleet-Health servisine iletir.

- Fleet-Health (FastAPI): Gelen alarm verilerini işleyerek sistemin genel sağlık durumunu tek bir özet haline getirir.
- Debug Scrape Controller: Alertmanager'ı 30 saniyede bir sorgulayan Python tabanlı otomasyondur. Kritik bir alarm durumunda *prometheus.yml* üzerindeki engelleme (DROP) kurallarını kaldırarak detaylı analiz metriklerinin akmasını sağlar.

İki Aşamalı Kazıma Stratejisi (Two-Tier Scraping):

1. Alertmanager servisini her 30 saniyede bir sorgular.
2. prometheus.yml dosyasındaki metrik engelleme (DROP) kurallarını dinamik olarak ekler veya kaldırır.
3. /-/reload API ucu üzerinden Prometheus'u yeniden yükleyerek ayarları günceller
4. Konteyner kapatıldığında tüm engelleme kuralları devre dışı kalır ve tüm metrikler kısıtlama olmaksızın akmaya başlar.

17.3 Ön Koşullar

Sistemin sorunsuz çalışabilmesi için aşağıdaki altyapı ve yazılım gereksinimlerinin karşılanması gerekmektedir.

Gerekli Yazılımlar:

- Docker (Konteyner Çalışma Zamanı):
  - Windows/Mac: Docker Desktop üzerinden kurulmalıdır.
  - Linux: apt install docker.io komutu ile kurulur.
- Docker Compose (Çoklu Konteyner Yönetimi):
  - Windows/Mac: Docker Desktop içinde varsayılan olarak gelir.
  - Linux: apt install docker-compose komutu ile kurulur.
- Node.js 18+ (Backend API):
  - Halihazırda ana makine üzerinde çalışıyor olmalıdır.

Ağ Gereksinimleri:

Gelen Bağlantılar (Dış Erişim):

Arayüz veya API etkileşimi için bu portların erişime açık olması gerekir:

- Port 9090: Prometheus (Arayüz ve API erişimi)

- Port 9093: Alertmanager (Arayüz erişimi)
- Port 8080: Fleet Health Webhook (API erişimi)
- Port 8081: Debug Controller (Durum API erişimi)

Dahili Bağlantılar (Sadece Yerel):

Bu portlar servisler arası iletişim için kullanılır ve dışarıya açılması gerekmez:

- Port 9115: Blackbox Exporter
- Port 9104: MariaDB Exporter
- Port 9234: Mosquitto Exporter
- Port 9000: Docker Events Exporter

17.4 Kurulum Kılavuzu

Adım 1: Ortam Yapılandırması

Tüm servisler deploy/prometheus/ dizininde bulunan bir .env dosyasına dayanır.

- Dizin ve Düzenleme:
  - Dizine gidin: cd deploy/prometheus
  - Ortam dosyasını açın: nano .env
- Temel Değişken Grupları:
  - Bulut ve Backend: CLOUD_HOST, BACKEND_HOST ve BACKEND_PORT değişkenlerini tanımlayın.
  - Veri Tabanı (MariaDB): DB_HOST, DB_PORT, DB_USER ve MYSQLD_EXPORTER_PASSWORD bilgilerini girin.
  - Mesajlaşma (MQTT): MQTT_BROKER_HOST ve MQTT_BROKER_PORT bilgilerini ayarlayın.
  - Altyapı: DOCKER_HOSTNAME bilgisini tanımlayın.
  - Mantıksal Kontroller: POLL_INTERVAL, HYSTERESIS_MINUTES ve MAX_DEBUG_TTL_MINUTES değerlerini düzenleyin.

Adım 2: Sistemin Başlatılması

İzleme altyapısını başlatmak için aşağıdaki komutu çalıştırın:

- Komut: docker-compose up -d
- Aktif Servisler (Toplam 8):

    - Prometheus: Temel zaman serisi veri tabanı ve alarm motoru.
    - Alertmanager: Alarm yönlendirme ve bildirim gruplandırma işlemlerini yönetir.
    - Blackbox Exporter: HTTP, TCP ve DNS taramalarını (probes) gerçekleştirir.
    - MariaDB Exporter: Veri tabanı performans metriklerini toplar.
    - Mosquitto Exporter: MQTT broker sağlığını izler.
    - Docker Events Exporter: Konteyner yaşam döngüsü değişikliklerini takip eder.
    - Fleet Health: Alarmlar için webhook verilerini bir araya getirir.
    - Debug Controller: Metrikler için dinamik filtrelemeyi yönetir.

Adım 3: Doğrulama

Kurulumdan sonra sistemin sağlığını bu kontrollerle doğrulayın:

- Konteyner Durumu: Tüm konteynerlerin "Up" durumda olduğundan emin olmak için docker-compose ps komutunu kullanın.
- Prometheus Hedefleri: API üzerinden tüm toplayıcıların taranıp taranmadığını kontrol edin:
    - curl http://localhost:9090/api/v1/targets
- API Sağlığı: Fleet Health özetini kontrol edin: curl http://localhost:8080/summary
    - Debug Controller durumunu kontrol edin: curl http://localhost:8081/status

Adım 4: Backend Entegrasyonu

Backend servisi, izleme verileri ile kullanıcı arayüzü arasında bir köprü (proxy) görevi görür.

- Gerekli Yapılandırma: Backend .env dosyanızın FLEET_HEALTH_URL bilgisini içerdiğinden emin olun.
- İşlevsellik: /health/summary uç noktası verileri Fleet Health üzerinden çeker ve ön yüzdeki HealthContext bileşenine sunar.

Adım 5: Frontend Yapılandırması

Ön yüz, API ile iletişim kurmak için ortama özel URL'ler kullanır.

- Yerel Geliştirme: frontend/.env.development dosyasına REACT_APP_API_BASE_URL değerini girin.

- Üretim/Bulut: frontend/.env.production dosyasına bulut IP adresinizi girin.
- Kapsam: Bu ayarlar AdminPanel.js, HealthContext.js ve AuthContext.js dosyalarını etkiler.

17.5 Config Dosyaları

Dağıtım dizini; ortam yapılandırması, konteyner mantığı ve alarm tanımlarını birbirinden ayıracak şekilde düzenlenmiştir.

- Kök Yapılandırma:
  - .env: Tüm servisler tarafından paylaşılan merkezi ortam değişkenleri.
  - docker-compose.yml: 8 konteynerin tamamını tanımlayan ana orkestrasyon dosyası.
- Servis Yapılandırmaları:
  - prometheus-docker.yml: Tarama yapılandırmaları ve metrik yeniden etiketleme mantığı.
  - alertmanager-docker.yml: Yönlendirme, gruplandırma ve engelleme kuralları.
  - blackbox.yml: Tarama modülleri (HTTP, TCP, ICMP, DNS).
- Özel Python Servisleri:
  - fleet_health.py ve Dockerfile.fleet-health: Webhook birleştirme için FastAPI sunucusu.
  - debug_scrape_controller.py ve Dockerfile.debug-controller: Dinamik metrik filtreleme mantığı.
- Alarm Kuralları (/rules):
  - Backend (14), Frontend (9), MariaDB (13), MQTT (6), Docker (10) ve Filo/Ağ (23).

Prometheus Tarama Envanteri

Sistem, çeşitli görev türleri aracılığıyla hem dahili toplayıcıları hem de harici bağımlılıkları izler.

Core & Infrastructure Jobs

Job Name	Target	Purpose
prometheus	localhost:9090	Self-monitoring.
docker-daemon	host:9323	Docker Engine engine-level metrics.
docker-events-exporter	docker-events-exporter:9000	Tracking container lifecycle (start/stop/kill).
mariadb-exporter	mariadb-exporter:9104	Database performance and health.
mosquitto-exporter	mosquitto-exporter:9234	MQTT broker throughput and client counts.

Application & Connectivity Probes

Job Name	Target	Purpose
backend-app	host:3001	Express API metrics (Node.js).
frontend-metrics	host:3001	Metrics gathered from the React SPA.
blackbox-internet-*	Google/Cloudflare/DNS	Validating outbound internet connectivity.
blackbox-backend-health	host:3001/health	Validating Backend availability via HTTP probe.
blackbox-algorithm-health	8 specific endpoints	Monitoring specific algorithm availability.

Alertmanager ve Yönlendirme Mantığı

Sistem, akıllı gruplandırma ve engelleme yöntemleriyle alarm yorgunluğunu önleyecek şekilde yapılandırılmıştır.

Bildirim Yönlendirme:

- Birleştirme: Alarmlar alarm adına, servise ve alt sisteme göre gruplandırılır.
- Aciliyet Seviyeleri:
  - Kritik: 5 dakikalık tekrarlama aralığı (Yüksek öncelik).
  - Uyarı: 10 dakikalık tekrarlama aralığı.
- Hedef: Tüm alarmlar, 8080 portundaki fleet-health webhook servisine HTTP POST aracılığıyla iletilir.

Engelleme (Gürültü Azaltma):

Kök neden zaten belirlenmişken ikincil bildirimler altında boğulmamanız için; bir servis (İnternet, Backend, MariaDB vb.) için Kritik bir alarm aktifse, o servisle ilgili tüm Uyarı seviyesindeki alarmlar durdurulur.

17.6 Admin Panel Sağlık İzleme Sekmesi (Health Monitoring Tab)

Sağlık Durumu Veri Akışı

Sistem, verileri toplayıcılardan kullanıcı tarayıcısına taşımak için katmanlı bir mimari kullanır:

1. Kaynak: Fleet-Health (:8080) aktif alarmları bir araya getirir.
2. Vekil (Proxy): Backend (:3001) özet verileri çeker ve temizler.
3. İstemci: React uygulamasındaki HealthContext.js, backend servisini her 3 saniyede bir sorgular.
4. Görüntüleme: Durum bilgisi AlertBar ve AdminPanel bileşenlerine dağıtılır.

Kullanıcı Arayüzü Bileşen Detayları

1. HealthContext.js

- Durum Yönetimi: Sisteme küresel bir durum atar: Yeşil (Sağlıklı), Sarı (Uyarı) veya Kırmızı (Kritik).
- Hata Yönetimi: Backend servisinin çevrimdışı olduğunu tespit eder ve sistem genelinde "Backend erişilemiyor" uyarısını tetikler.
- Kullanıcı Tercihleri: Alarm susturma (mutedAlerts) ve görünürlük ayarlarını yönetir.

2. AlertBar.js

- Kalıcılık: Tüm panel sayfalarında görünen yapışkan bir üst bilgi çubuğudur.
- Görsel İpuçları: Dikkat çekmek için yeni bir alarm tespit edildiğinde 5 saniye boyunca yanıp söner.
- Gezinme: Bir kısayol görevi görür; çubuğa tıklandığında kullanıcıyı inceleme yapması için doğrudan Yönetim Paneline yönlendirir.

3. Yönetim Paneli İzleme Özellikleri Panel, sistem sağlığını yönetmek için iki özel sekmeye ayrılmıştır:

- Sistem Alarmları Yönetimi:
    - Takip: Geçmiş alarmların kalıcı bir geçmişini tutmak için tarayıcı yerel depolamasını (localStorage) kullanır.
    - İşlemler: Kullanıcıların raporlama için alarm verilerini ertelemesine, filtrelemesine veya dışa aktarmasına (JSON/CSV) olanak tanır.

- İzleme Sistemleri:
    - Görselleştirme: 6 temel sistem (Backend, Frontend, DB, MQTT, Docker, Filo) için Recharts tabanlı gerçek zamanlı grafikler sunar.
    - Eşik Değerler: Alarm kuralı eşiklerini görüntülemek ve düzenlemek için doğrudan erişim sağlar.
    - Kontroller: Zaman aralığı seçimi (24 saate kadar) ve manuel otomatik yenileme düğmesi içerir.
    - Satır İçi Düzenleyici: Eşik değerleri doğrudan arayüz üzerinden değiştirmek için kural düzenleyici sunar.

Ortam Yapılandırması

Ön yüz, tüm ağ istekleri için tek bir ortam değişkenine başvurarak taşınabilirliğini korur:

// Merkezi API Temeli const API_BASE = process.env.REACT_APP_API_BASE_URL;

Not: Bu değişkenler derleme (build) sürecinde koda işlendiğinden, ön yüzü derlemeden önce .env.development (veya production) dosyanızın doğru bulut IP adresiyle güncellendiğinden emin olun.

17.7 Alert Tanımları

Backend Alerts (14 total)

Alert Name	Severity	Trigger	Duration	Description
Backend_Up	critical	up{job=~".backend."} == 0	1m	Backend is unreachable
Backend_HTTP_Request_Duration_High	warning	P95 latency greater than 500ms	5m	High request latency
Backend_HTTP_Request_Rate_High	warning	greater than 50 req/s	5m	Unexpected traffic spike
Backend_HTTP_Request_Rate_Low	warning	less than 0.01 req/s	30m	Traffic drop (upstream issue)
Backend_HTTP_Error_Rate_High	critical	5xx rate greater than 5 percent	5m	High error rate
Backend_HTTP_4xx_Errors_Count	warning	greater than 5 errors/s	5m	Client errors (bad requests)
Backend_Response_Size_Large	warning	P95 size greater than 1MB	5m	Large response payloads

Backend_CPU_Usage_High	critical	CPU greater than 80 percent	5m	High CPU usage
Backend_Memory_Usage_High	warning	Memory greater than 80 percent	5m	High memory usage
Backend_Heap_Usage_Critical	critical	Heap greater than 96 percent	3m	Heap critically full
Backend_GC_Duration_High	warning	GC greater than 100ms	5m	Slow garbage collection
Backend_GC_Frequency_High	warning	GC greater than 10/min	5m	Frequent GC cycles
Backend_Event_Loop_Lag_High	critical	Lag greater than 200ms	5m	Event loop blocked
Backend_Active_Handles_High	warning	Handles greater than 1000	5m	Resource leak

Frontend Alerts (9 total)

Alert Name	Severity	Trigger	Duration	Description
Frontend_Metrics_Endpoint_Down	warning	up == 0	5m	Metrics endpoint unreachable
Frontend_App_Load_Time_High	warning	P95 load time greater than 3s	5m	Slow page loads

Frontend_TTFB_High	warning	P95 TTFB greater than 500ms	5m	High time to first byte
Frontend_TTFCP_High	warning	P95 TTFCP greater than 2s	5m	Slow first contentful paint
Frontend_DOMContentLoaded_Slow	warning	P95 DCL greater than 3s	5m	Slow DOM load
Frontend_SPA_Navigation_Slow	warning	P95 nav greater than 5s	5m	Slow navigation
Frontend_Time_To_Interactive_High	critical	P95 TTC greater than 10s	5m	Extremely slow content
Frontend_Time_To_Usable_High	warning	P95 TTI greater than 4s	5m	Slow interactivity
Frontend_High_Bounce_Rate	warning	Bounce rate greater than 50%	10m	High bounce rate

MariaDB Alerts (13 total)

Alert Name	Severity	Trigger	Duration	Description
MariaDB_Node_Down	critical	probe_success == 0	1m	TCP port unreachable
MariaDB_Node_Slow	warning	Query rate less than 0.12/s	1m	Very low query rate

MariaDB_Down	critical	mysql_up == 0	30s	MySQL exporter connection failed
MariaDB_Restarted	info	Uptime less than 60s	0m	Recent restart detected
MariaDB_High_Connections	warning	Connections greater than 80 percent	1m	High connection usage
MariaDB_High_Threads_Running	warning	Running greater than 60 percent	1m	Many active queries
MariaDB_High_QPS	warning	QPS greater than 1000	1m	High query rate
MariaDB_Slow_Queries	warning	Slow queries greater than 0.05/s	1m	Queries exceeding threshold
MariaDB_High_Prepared_Statements	warning	Prepared stmts greater than 1000	5m	Many prepared statements
MariaDB_InnoDB_Log_Waits	warning	Log waits greater than 0	5m	InnoDB log buffer waits
MariaDB_Too_Many_Open_Files	warning	Open files greater than 80 percent	5m	File handle exhaustion
MariaDB_InnoDB_History_Length	warning	History greater than 10000	5m	Long transaction history

MariaDB_InnoDB_Force_Recovery	critical	Force recovery greater than 0	1m	InnoDB in recovery mode

MQTT Alerts (6 total)

Alert Name	Severity	Trigger	Duration	Description
MQTT_Exporter_Down	critical	up == 0	2m	MQTT exporter unreachable
MQTT_Broker_TCP_Unreachable	critical	probe_success == 0	2m	MQTT broker port unreachable
MQTT_Connections_Maximum_Reached	critical	Connections greater than 95 percent	5m	Near connection limit
MQTT_Messages_Dropped	critical	Dropped greater than 0	2m	Messages dropped by broker
MQTT_Clients_Expired	warning	Expired greater than 0	5m	Client session expiry
MQTT_Retained_Messages_Growing	warning	Growth greater than 100/10m	10m	Retained message explosion

Docker Alerts (10 total)

Alert Name	Severity	Trigger	Duration	Description
Docker_Daemon_Down	critical	up == 0	30s	Docker daemon unreachable
Docker_Daemon_High_Event_Rate	warning	Events greater than 10/s	2m	High Docker event rate
Docker_No_Running_Containers	warning	Running == 0	5m	No containers running
Docker_High_Container_Start_Rate	warning	Starts greater than 2/s	2m	Frequent container restarts
Docker_Container_Bad_ExitCode	critical	Exit code != 0	1m	Container crashed
Docker_Container_OOM_Event	critical	OOM event detected	1m	Container killed by OOM
Docker_Many_Container_Deaths	warning	Deaths greater than 5/5m	5m	Multiple container failures
Docker_Events_Exporter_Down	warning	up == 0	2m	Events exporter unreachable
Docker_Container_Health_Check_Failing	warning	Health check failed for 5m	2m	Health checks are failing

Docker_Container_Health_Check_Spike	warning	Health check fails greater than 10 for 10m	1m	More than 10 health check failures

17.8 Debug Scrape Kontrolörü

Operasyonel Mantık

Kontrolör, aşağıdaki yaşam döngüsüne sahip 30 saniyelik sürekli bir sorgulama döngüsünde çalışır:

1. Sorgulama: Aktif kritik alarmlar için Alertmanager'ı kontrol eder.
2. Eşleştirme: Tetiklenen alarmları önceden tanımlanmış Tetikleyici Listesi ile karşılaştırır.
3. Etkinleştirme:
   - Bir eşleşme bulunursa, Prometheus yapılandırmasından DROP (engelleme) kurallarını kaldırır.
   - Prometheus'a bir POST /-/reload isteği gönderir.
   - Sonuç: Detaylı hata ayıklama metrikleri veri tabanına akmaya başlar.
4. Devre Dışı Bırakma (Histerezis):
   - Alarmlar çözüldüğünde, kararlılığı sağlamak için 10 dakikalık bir soğuma süresi (Histerezis) bekler.
   - DROP kurallarını tekrar ekler ve Prometheus'u yeniden yükler.
5. Güvenlik Ağı: 30 dakikalık Maksimum TTL, hata ayıklama modunun süresiz olarak açık kalmamasını sağlayarak depolama alanını korur.
6. Hata Güvenliği: Konteyner durursa, kontrolörün kendi sorunlarını giderme sırasında veri kaybı yaşanmaması için tüm kısıtlamaları kaldırır.

Servis Tetikleyici Eşleştirmesi

Bu tablo, hangi alarmların belirli yüksek kardinaliteli metriklerin kilidini açacağını tanımlar:

Service	Trigger Alerts (Examples)	Debug Metrics Unlocked

Backend	Backend_Up, Heap_Usage_Critical	GC details, request durations, active handles.
Frontend	App_Load_Failure, TTI_High	Navigation timing, resource details, memory stats.
MariaDB	MariaDB_Down, InnoDB_Recovery	Slow queries, InnoDB internals, handler/sort stats.
MQTT	Connections_Maxed, Messages_Dropped	Client states, byte counters, detailed subscriptions.
Docker	Daemon_Down, Container_OOM	Builder cache, image actions, Go runtime metrics.
Fleet	Internet_Down, DNS_TCP_Down	ICMP/DNS timing breakdowns, SSL/IP protocol details.

17.9 Operasyonlar ve Komutlar

Docker Compose İşlemleri:

8 servisten oluşan izleme yığınının yaşam döngüsünü yönetmek için kullanılan standart komutlardır.

- Dağıtım:
  - docker-compose up -d — Tüm yığını arka planda çalışacak şekilde başlatır.
  - docker-compose down — Tüm konteynerleri ve ağları durdurur ve kaldırır.
- Bakım:
  - docker-compose restart <servis_adı> — Diğerlerini etkilemeden tek bir servisi (örneğin prometheus) yenilemek için kullanılır.
  - docker-compose ps — Tüm konteynerlerin mevcut durumunu, port eşleşmelerini ve sağlık durumlarını görüntüler.

- Sorun Giderme:
  - docker-compose logs -f <servis_adı> — Gerçek zamanlı günlük akışını izler (özellikle fleet-health veya debug-controller hatalarını ayıklamak için önerilir).

Prometheus API Komutları:

Web arayüzünü kullanmadan zaman serisi veri tabanıyla doğrudan etkileşime girmek için bu uç noktaları kullanın.

- Yapılandırma Yönetimi:
  - curl -X POST http://xxx.xxx.xx.x:9090/-/reload — Prometheus'u yeniden başlatmaya gerek kalmadan prometheus-docker.yml dosyasındaki veya alarm kurallarındaki değişiklikleri uygulamaya zorlar.
- Metrik Keşfi:
  - curl http://xxx.xxx.xx.x:9090/api/v1/targets — Tüm tarama hedeflerinin ve mevcut durumlarının (Açık/Kapalı) JSON listesini döndürür.
- Veri Sorgulama:
  - Anlık Sorgu: curl 'http://xxx.xxx.xx.x:9090/api/v1/query?query=up' (Servislerin hayatta olup olmadığını kontrol eder).
  - Aralık Sorgusu: Belirli bir zaman penceresindeki (örneğin 12 saatlik bir dönem) geçmiş verileri almak için kullanılır.

Sağlık Kontrolü Uç Noktaları

Bu komutlar, ham alarmdan nihai vekil sunucuya (proxy) kadar veri hattını doğrulamanıza olanak tanır.

Component	Command	Purpose
Fleet Health	curl http://xxx.xxx.xx.x:8080/summary	Verify the aggregator is receiving and processing alerts.
Backend Proxy	curl http://	Verify the Node.js API can

	xxx.xxx.xx.x:3001/health/summary	successfully talk to Fleet Health.
Debug Controller	curl http:// xxx.xxx.xx.x 8081/status	Check if any "DROP" rules are currently active or if debug mode is triggered.
Alertmanager	curl http:// xxx.xxx.xx.x:9093/api/v2/alerts	See the raw active alert objects before they are processed.

17.10 Sorun Giderme

AlertBar içerisinde "Health endpoint unreachable" (Sağlık uç noktasına ulaşılamıyor) uyarısı:

Bu durum, Backend servisinin fleet-health servisiyle iletişim kuramadığı anlamına gelir.

- Doğrulama Adımları:
  - Konteynırın çalıştığını doğrulayın: docker-compose ps fleet-health
  - Toplayıcıyı doğrudan test edin: curl http://xxx.xxx.xx.x:8080/summary
  - Backend servisinin ortam değişkenini kontrol edin: grep FLEET_HEALTH_URL ~/app/backend/.env
- Çözüm: Backend .env dosyasının FLEET_HEALTH_URL=http://xxx.xxx.xx.x:8080 değerini içerdiğinden emin olun ve backend servisini yeniden başlatın.

Prometheus Hedefi "DOWN" (Kapalı):

Prometheus belirli bir toplayıcıdan (exporter) metrikleri çekemiyor demektir.

Hızlı Tanılama:
Bash
```
curl -s http://xxx.xxx.xx.x:9090/api/v1/targets | jq '.data.activeTargets[] |
select(.health=="down") | {job: .labels.job, target: .scrapeUrl,
error: .lastError}'
```

-
- Yaygın Çözümler:
    - Linux Ağ Ayarları: Eğer host.docker.internal çözümlenemiyorsa, Docker köprü IP'sini (bridge IP) kullanın: 172.17.0.1.
    - Docker Daemon: daemon.json dosyasında "metrics-addr": "0.0.0.0:9323" ve "experimental": true ayarlarının etkin olduğundan emin olun.
    - Çalışmayan Konteynırlar: Çöken toplayıcıları canlandırmak için docker-compose up -d komutunu çalıştırın.

Filtreleme Etkin Olmasına Rağmen Hata Ayıklama (Debug) Metrikleri Hala Görünüyor:

Kullanıcılar genellikle grafikleri hala gördükleri için filtrenin başarısız olduğunu düşünür.

- Neden: Prometheus bir zaman serisi veri tabanıdır. DROP kuralları yeni veriyi durdurur ancak geçmiş veriler görüntüleme aralığından çıkana kadar kalır.
- Doğrulama: curl üzerinden anlık bir sorgu çalıştırın. Sonuç boşsa filtre çalışıyor demektir. Zaman penceresi ilerledikçe grafikler sonunda düzleşecek ve kaybolacaktır.

Konteynır Başlamıyor: Genellikle yapılandırma hatalarından veya güncel olmayan derlemelerden kaynaklanır.

- Günlükleri Kontrol Et: docker-compose logs debug-controller

Yeniden Derlemeye Zorla: Eğer Python betiklerini veya requirements.txt dosyasını değiştirdiyseniz:
Bash
```
docker-compose build debug-controller

docker-compose up -d debug-controller
```

Frontend Alarmlar Görünmüyor:

Eğer Prometheus alarmları tetikliyor ancak panel boş görünüyorsa, şu veri yolunu takip edin:

- Alertmanager: http://xxx.xxx.xx.x:9093 adresini kontrol edin (Alarm gerçekten tetikleniyor mu?).
- Fleet-Health: curl http://xxx.xxx.xx.x:8080/summary (Webhook ulaştı mı?).
- Backend Proxy: curl http://xxx.xxx.xx.x:3001/health/summary (Vekil sunucu çalışıyor mu?).
- Ön Yüz Yapılandırması: Değişken adının doğruluğunu kontrol edin: REACT_APP_API_BASE_URL (yaygın bir yazım hatası olan REACT_APP_API_BASE olmadığını teyit edin).
- IP/URL adresinin ortamınızla (Geliştirme veya Üretim) eşleştiğini doğrulayın.

18. Sistem Geneli Uçtan Uca Veri Akışı

18.1 Veri Girişi - Araç Telemetrisi

Adım 1: Gerçek Araç Verisi

```
Araç Sensörleri → MQTT (0.0.0.0:1885)
        → Topic: /json/secret/{vehicle_id}/attrs
        → socket_listener.js
        → MariaDB (vehicle_tracking_fiware)
```

Adım 2: Simülasyon Verisi

```
SUMO TraCI → function.py → fiware_queue
    → MQTT Publisher → Backend
    → MariaDB (vehicle_tracking_fiware)
```

18.2 Rota Planlama Akışı

```
Frontend (Route Optimization)
  → XML dosyaları yükle
  → Backend /rml/* endpoint'leri
  → Algoritma çalıştır (SA/TS/ALNS)
  → Route4Vehicle.json üret
  → Route4Sim.xml üret
  → watchRouteSync.js
  → SUMO/ESOGU_SUMO/Route4Sim.xml
```

18.3 Simülasyon Çalıştırma

```
Frontend "Start Simulation"
  → Backend /api/sumo (proxy)
  → SUMO main.py /sumo
  → function.sumo()
  → TraCI simülasyon döngüsü
  → FIWARE yayını
  → Backend socket_listener.js
  → MongoDB alerts
  → Frontend güncellemesi
```

18.4 Tahmin Servisleri Akışı

```
Fleet Monitoring "Predict" butonu
```

→ /api/predict (machine/app.py)

→ CatBoost model.predict()

→ SHAP açıklamaları

→ Menzil hesaplama

→ Frontend görselleştirme

```
```

18.5 A* Rota Hesaplama Akışı

```
```

Fleet Monitoring "Calculate Depot Route"

→ route-service /api/fiware/calculate-depot-route

→ OSMnx graf yükleme

→ Edge tabanlı A*

→ JSON yanıt (edge listesi)

→ Backend /api/astar/add-route

→ Route4Vehicle.json güncelleme

→ Frontend harita çizimi

```
```

18.6 Uyarı Sistemi Akışı

```
```

Araç verisi alınır (socket_listener.js)

→ generateDynamicAlerts()

→ Eşik kontrolü (batarya, voltaj, sıcaklık...)

→ MongoDB Alert kaydı

→ Frontend /api/alerts

→ AlertPanel görselleştirme

→ Alert bildirimi

```

## 18.7 Sağlık İzleme Akışı

Frontend "Admin Panel"
  → Metrik Üretimi (Exporter)
  → Metrik Toplama (ServiceMonitor → Prometheus)
  → Alert Değerlendirmesi ( PrometheusRule → Alertmanager)
  → Webhook İletimi (Alertmanager → Fleet Health)
  → Debug Scraping (Debug Controller)
  → Backend Proxy (Express.js)
  → Frontend Polling (HealthContext)
  → Alert Yönetimi & Görselleştirme (AlertBar + AdminPanel)
```

19. Yeni Geliştiriciler İçin Özet

19.1 Projeye Başlangıç Rehberi

Önce Okunması Gereken Dosyalar:

1. `.env` - Ortam değişkenleri ve bağlantı bilgileri
2. `start_all.bat` - Tüm servislerin başlatılma sırası
3. `backend/server.js` (ilk 200 satır) - API yapısı
4. `frontend/src/App.js` - Uygulama yapısı ve rotalar
5. `frontend/src/services/api.js` - API endpoint tanımları

Çalıştırma Sırası:

1. MariaDB ve MongoDB erişilebilir olmalı
2. MQTT broker çalışıyor olmalı (port 1885)
3. `start_all.bat` çalıştırılır veya:

Backend: `cd backend && node server.js`

SUMO: `cd Sumo && python main.py`

Machine: `cd machine && python app.py`

Route-Service: `cd route-service && python app.py`

Frontend: `cd frontend && npm start`

19.2 Güvenle Müdahale Edilebilecek Dosyalar

`frontend/src/pages/*.js` | Düşük | UI değişiklikleri

`frontend/src/components/*.js` | Düşük | Bileşen güncellemeleri

`frontend/src/data/*.js` | Düşük | Statik veri dosyaları

`backend/routes/*.js` | Orta | API endpoint'leri

`backend/models/*.js` | Orta | Şema değişiklikleri

`deploy/monitoring/*.yaml` | Düşük | Monitoring kuralları

19.3 Kritik Dosyalar

`backend/server.js` | Ana giriş noktası, dikkatli değişiklik gerektirir

`backend/socket_listener.js` | Gerçek zamanlı veri akışı, uyarı sistemi

`backend/db.js` | Veritabanı bağlantıları

`Sumo/function.py` | SUMO simülasyon mantığı

`machine/federated_training.py` | FL eğitim döngüsü

`route-service/osmnx_astar_edge_based.py` | A* algoritma implementasyonu

19.4 Geliştirmeye Güvenli Başlama Önerileri

1. Yerel Ortam Kurulumu

Node.js 18+, Python 3.9+

MariaDB yerel kurulum veya uzak bağlantı

MongoDB Atlas veya yerel kurulum

2. İlk Test

Backend başlat: `node server.js`

`/api/test-delivery` endpoint'ini test et

Frontend başlat: `npm start`

Login sayfasını kontrol et

3. Küçük Değişiklikle Başla

UI metinleri değiştir

Yeni bir basit API endpoint'i ekle

Mevcut bir bileşene küçük özellik ekle

4. Loglama Kullan

Backend: `console.log()` ifadeleri

Frontend: React DevTools

SUMO: `SUMO_VERBOSE=true`

5. Git Branch Stratejisi

Ana branch'e direkt push yapma

Feature branch oluştur

Pull request ile birleştir

20. Database İşlemleri Localhost için

database: localde çalıştırırken

fleetmanagementdb.sql dosyasını run sql script ile çalıştırdığınızda fleetmanagementdb olarak database otomatik oluşturacaktır, tabloları içinde bulunan veriler ile oluşturacaktır

mariadb ve heidisql add or remove application üzerinden uninstall yaparak yeniden kurduktan sonra bilgisayarınızda my.ini dosyasını aratarak 0.0.0.0 bilgisinin temizlendiğinden emin olmanız gerekmektedir.

uyumluluk sorunlarını gidermek için, **MariaDB 11.3.2** versiyonunu yüklemeniz hataların önüne geçecektir.

https://mariadb.com/docs/platform/post-download/mariadb-server-11.3.2

HeidiSQL'in en güncel versiyonu için;

https://www.heidisql.com/download.php

[mysqld]

datadir=C:/Program Files/MariaDB 11.3/data

port=3306

innodb_buffer_pool_size=1003M

[client]

port=3306

plugin-dir=C:\Program Files\MariaDB 11.3/lib/plugin

database bağlantısını sıfırdan kurarken (mariadb install aşamasında kullanıcı adı: root ve belirlediğiniz şifreyi **filo yönetimi uygulamasında**

.env C:\Users\rootr\Desktop\applications\fmrevize005\app\.env (örnek path) burada ise 1 tane mevcut.

ve C:\Users\rootr\Desktop\applications\fmrevize005\app\Sumo\function.py (örnek path) iki tane mariadb connection bilgisi mevcut local mariadb de root user'ı için belirlediğiniz parola ile değiştirmelisiniz.

```
db_connection = MySQLdb.connect(
        host="127.0.0.1",
        user="remoteuser",
        password="arm2028",
        database="fleetmanagementdb
```

MariaDB Veri Aktarımı ve Kullanıcı Yetkilendirme için;

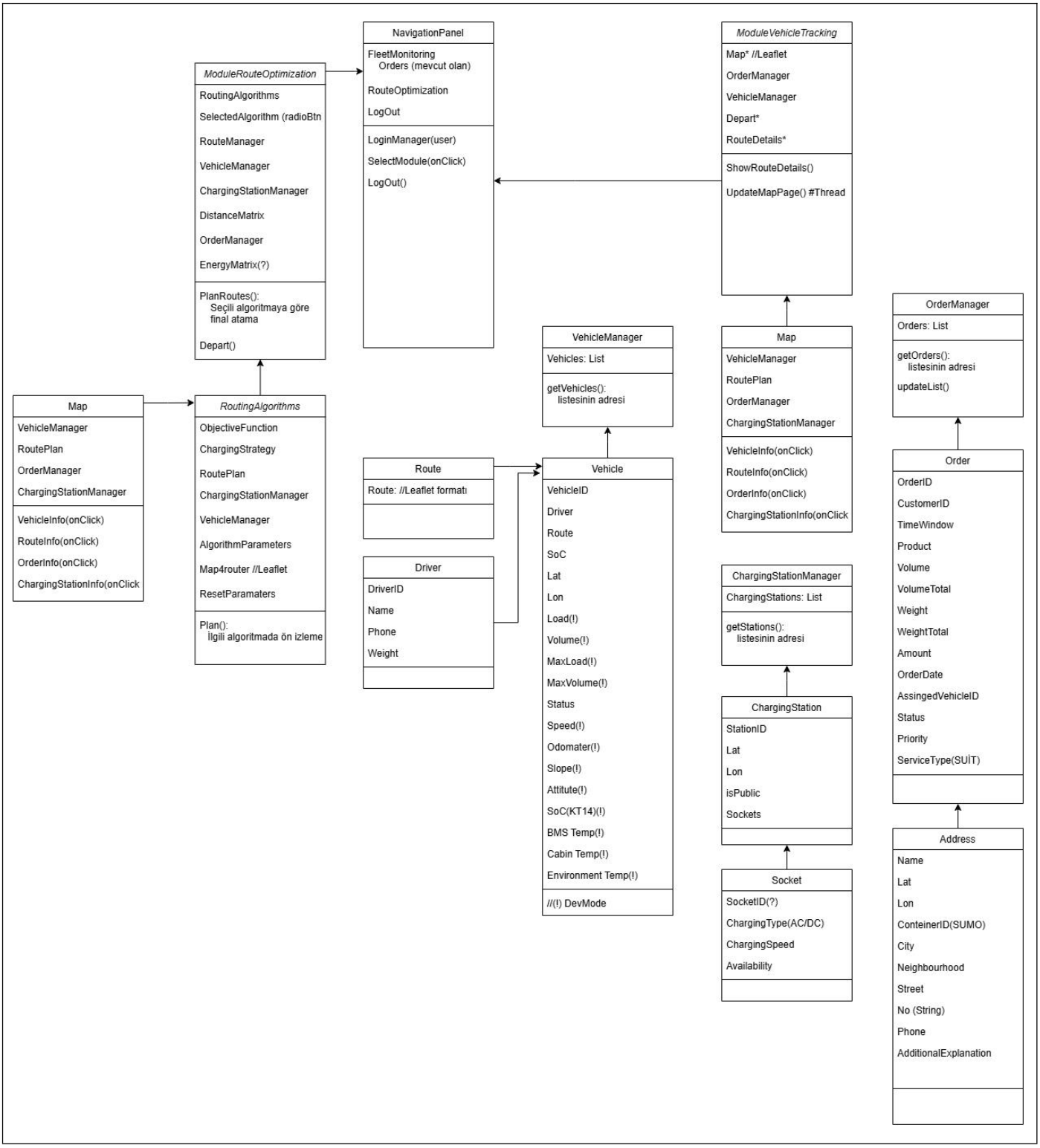
- fleetmanagementdb.sql dosyası HeidiSQL üzerinden içe aktarılmıştır.
- Uzaktan erişim ve uygulama bağlantıları için aşağıdaki kullanıcı oluşturulmuştur:

SQL Kullanıcı Oluşturma
CREATE USER 'remoteuser'@'%' IDENTIFIED BY 'arm2028'; GRANT ALL PRIVILEGES ON *.* TO 'remoteuser'@'%' WITH GRANT OPTION; FLUSH PRIVILEGES;

Remoteuser kullanıcısı yukarıda paylaşılan SQL kodu ile oluşturulabilir ve tüm yetkileri tam olarak verilir.

21. ER Şeması

MariaDB Database ER şeması İlişkisel 21.02.2025 Son güncelleme tarihi


NavigationPanel
FleetMonitoring
Orders (mevcut olan)
RouteOptimization
LogOut
LoginManager(user)
SelectModule(onClick)
LogOut()
ModuleVehicleTracking
Map* //Leaflet
OrderManager
VehicleManager
Depart*
RouteDetails*
ShowRouteDetails()
UpdateMapPage() #Thread
ModuleRouteOptimization
RoutingAlgorithms
SelectedAlgorithm (radioBtn
RouteManager
VehicleManager
ChargingStationManager
DistanceMatrix
OrderManager
EnergyMatrix(?)
PlanRoutes():
Seçili algoritmaya göre
final atama
Depart()
OrderManager
Orders: List
getOrders():
listesinin adresi
updateList()
VehicleManager
Vehicles: List
getVehicles():
listesinin adresi
Map
VehicleManager
RoutePlan
OrderManager
ChargingStationManager
VehicleInfo(onClick)
RouteInfo(onClick)
OrderInfo(onClick)
ChargingStationInfo(onClick
Map
VehicleManager
RoutePlan
OrderManager
ChargingStationManager
VehicleInfo(onClick)
RouteInfo(onClick)
OrderInfo(onClick)
ChargingStationInfo(onClick
RoutingAlgorithms
ObjectiveFunction
ChargingStrategy
RoutePlan
ChargingStationManager
VehicleManager
AlgorithmParameters
Map4router //Leaflet
ResetParamaters
Plan():
İlgili algoritmada ön izleme
Route
Route: //Leaflet formatı
Driver
DriverID
Name
Phone
Weight
Vehicle
VehicleID
Driver
Route
SoC
Lat
Lon
Load(!)
Volume(!)
MaxLoad(!)
MaxVolume(!)
Status
Speed(!)
Odomater(!)
Slope(!)
Attitute(!)
SoC(KT14)(!)
BMS Temp(!)
Cabin Temp(!)
Environment Temp(!)
//(!) DevMode
ChargingStationManager
ChargingStations: List
getStations():
listesinin adresi
ChargingStation
StationID
Lat
Lon
isPublic
Sockets
Socket
SocketID(?)
ChargingType(AC/DC)
ChargingSpeed
Availability
Order
OrderID
CustomerID
TimeWindow
Product
Volume
VolumeTotal
Weight
WeightTotal
Amount
OrderDate
AssingedVehicleID
Status
Priority
ServiceType(SUİT)
Address
Name
Lat
Lon
ConteinerID(SUMO)
City
Neighbourhood
Street
No (String)
Phone
AdditionalExplanation

22. MariaDB Veritabanı

İlişkisel Veritabanı

address	0	48.0 KiB	2025-11-28 18:34:11
algorithm_parameters	9	32.0 KiB	2025-11-28 18:34:11
algorithms	4	16.0 KiB	2025-11-28 18:34:11
charging_usage	40	48.0 KiB	2025-11-28 18:34:11
chargingstation	10	32.0 KiB	2025-11-28 18:34:11
customer	9	64.0 KiB	2025-11-28 18:34:11
customer_feedback	0	32.0 KiB	2025-11-28 18:34:11
customer_order_report	70	16.0 KiB	2025-11-28 18:34:11
delivery_logs	20	64.0 KiB	2025-11-28 18:34:11
driver	10	48.0 KiB	2025-11-28 18:34:11
driver_performance_report	130	32.0 KiB	2025-11-28 18:34:11
driver_success_report	130	16.0 KiB	2025-11-28 18:34:11
error_logs	107	64.0 KiB	2025-11-28 18:34:11
map	15	64.0 KiB	2025-11-28 18:34:11
order	293	96.0 KiB	2025-11-28 18:34:11
order_driver_performance	0	48.0 KiB	2025-11-28 18:34:11
order_product	0	48.0 KiB	2025-11-28 18:34:11
orders_mongodb	26	16.0 KiB	2025-11-28 18:34:11
product	20	16.0 KiB	2025-11-28 18:34:11
route_history	54	80.0 KiB	2025-11-28 18:34:12
route_optimization	53	48.0 KiB	2025-11-28 18:34:12
route_planned	134	80.0 KiB	2025-11-28 18:34:12
simulation_runs	4	16.0 KiB	2025-11-28 18:34:12
system_performance	8,043	368.0 KiB	2025-11-28 18:34:12
upcomingmaintenancealert			
users	19	112.0 KiB	2025-11-28 18:34:12
vehicle_info	5	64.0 KiB	2025-11-28 18:34:12
vehicle_logs_detail	0	32.0 KiB	2025-11-28 18:34:12
vehicle_maintenance_report	11	32.0 KiB	2025-11-28 18:34:12
vehicle_tracking	0	80.0 KiB	2025-11-28 18:34:12
vehicle_tracking_fiware	80,789	46.1 MiB	2025-11-28 18:34:12
vehicle_tracking_with_routes	2,283	304.0 KiB	2025-11-28 18:34:20
waypoints	0	48.0 KiB	2025-11-28 18:34:20

Bu bölüm, **ESOGU FMS** kullanılan **MariaDB tabanlı veritabanı mimarisini** ve sistemin veri yönetim yaklaşımını göstermektedir. Veritabanı, filo yönetimi, rota optimizasyonu, enerji analizi ve performans izleme süreçlerini destekleyecek şekilde **modüler ve ilişkisel** bir yapı ile tasarlanmıştır. Temel tablolar; **müşteri (customer)**, **sipariş (order, order_product, orders_mongodb)**, **sürücü (driver)** ve **araç (vehicle_info, vehicle_tracking, vehicle_tracking_fiware)** verilerini saklayarak operasyonel sürecin omurgasını oluşturmaktadır. Özellikle vehicle_tracking_fiware tablosu, FIWARE altyapısından gelen yüksek hacimli gerçek zamanlı araç telemetri verilerini tutarak sistemin canlı izleme kabiliyetini desteklemektedir. Bunun yanı sıra **rota ve optimizasyon** süreçlerine yönelik olarak route_optimization, route_planned, route_history, waypoints ve simulation_runs tabloları bulunmaktadır; seçilen algoritmalar ve parametreler algorithms ve algorithm_parameters tabloları üzerinden yönetilebilir (XML verileri için). **Enerji ve şarj yönetimi** tarafında ise chargingstation ve charging_usage tabloları, araçların şarj altyapısı ile etkileşimini ve enerji tüketim geçmişini kayıt altına almaktadır.

Operasyonel kaliteyi izlemek amacıyla delivery_logs, driver_performance_report, driver_success_report, vehicle_maintenance_report ve system_performance gibi tablolar sayesinde teslimat başarımı, sürücü performansı, bakım süreçleri ve sistem metrikleri detaylı biçimde analiz edilebilmektedir. Ayrıca error_logs tablosu ile sistem genelinde oluşan hatalar izlenebilir hâle getirilmiş, users tablosu ile rol tabanlı kullanıcı yönetimi desteklenmiştir. Görünüm (VIEW) olarak tanımlanan upcomingmaintenancealert yapısı ise bakım gereksinimlerini özetleyerek karar vericilere erken uyarı mekanizması sunmaktadır. Genel olarak bu MariaDB şeması, **yüksek hacimli veriyi yönetebilen, ölçeklenebilir, izlenebilir ve analitik karar destek süreçlerini besleyen** sağlam bir veritabanı altyapısı sunmaktadır.

23. MongoDB Veritabanı

Connection String Bilgileri
Connection info **Stats** 4 DBs 34 Collections **Hosts** cluster0-shard-00-01.on8m7.mongodb.net:27017 cluster0-shard-00-02.on8m7.mongodb.net:27017 cluster0-shard-00-00.on8m7.mongodb.net:27017 **Cluster** Replica Set atlas-nq1t4v-shard-0 3 Nodes **Edition** MongoDB 8.0.17 Atlas

mongodb+srv://fleetmanager:oQKOn0EFYeUQ6rmF@cluster0.on8m7.mongodb.net/RouteManagementDB?retryWrites=true&w=majority

mongodb+srv://fleetmanager:oQKOn0EFYeUQ6rmF@cluster0.on8m7.mongodb.net/RouteManagementDB?retryWrites=true&w=majority

MongoDB Compass uygulaması üzerinden connectionstring bilgileri girilerek MongoDB veritabanına erişim sağlanabilir. Manager olarak atlas admin sitesi de yönetilebilir. Ana hesap: opevafleetmanagement@gmail.com Şifre: FMapplication_1

MongoDB Collections

ESOGU Fleet Management System (ESOGU FMS) mimarisinde MongoDB, **yüksek hacimli, yarı-yapısal ve dinamik verilerin** yönetimi için konumlandırılmıştır. Sistem; gerçek zamanlı izleme, rota optimizasyonu, enerji hesaplamaları ve simülasyon çıktıları gibi ilişkisel olmayan veri yapıları ürettiğinden, MongoDB doküman tabanlı yaklaşımıyla MariaDB'yi tamamlayıcı bir rol üstlenmektedir. Bu kapsamda MongoDB altyapısı, **RouteManagementDB** ve **FleetManagement** olmak üzere iki ana veritabanı altında organize edilmiştir.

23.1 RouteManagementDB

RouteManagementDB, sistemin operasyonel ve karar destek süreçlerine doğrudan etki eden verilerin tutulduğu ana MongoDB veritabanıdır. Bu veritabanında; müşteri, sipariş, rota planları, araç enerjisi ve operasyonel performans bilgileri yer almaktadır.

23.1.1 Rota Optimizasyon Planları - route4plans

route4plans koleksiyonu, elektrikli araçlar için yürütülen rota optimizasyon senaryolarının **nihai sonuçlarını ve parametrelerini** saklamaktadır. Koleksiyon, her bir optimizasyon çalışmasını tek bir doküman altında tanımlayarak; problem tipi, hedef fonksiyon, şarj stratejisi ve performans metriklerini birlikte tutmaktadır.

Bu koleksiyonda yer alan temel alanlar şunlardır:

- **ProblemType**: Çözülen problem türü (ör. CEVRP Capacitated Electric Vehicle Routing Problem)
- **ObjectiveFunction**: Optimizasyon hedefi (MinDistance, MinEnergy, MinTime vb.)
- **ChargingStrategy**: Şarj politikası (FullCharge, PartialCharge vb.)
- **Araç parametreleri**: Batarya kapasitesi, maksimum hız, araç kütlesi, yük kapasitesi
- **PerformanceMeasure** alt dokümanı:
    - Toplam mesafe
    - Toplam süre
    - Toplam enerji tüketimi
    - Toplam gecikme
    - Şarj süresi
    - Ziyaret edilen düğüm sayısı
- **Routes**: Her bir aracın izleyeceği rota dizisi

Bu yapı sayesinde, bir optimizasyon senaryosuna ait **tüm girdiler ve çıktılar tek bir doküman** üzerinden izlenebilmektedir. Bu yaklaşım, algoritma karşılaştırmaları ve performans analizleri açısından büyük avantaj sağlamaktadır.

23.1.2 Araç Enerji Durumu /vehicleEnergy

vehicleEnergy koleksiyonu, her bir elektrikli aracın **anlık ve tarihsel enerji durumunu** izlemek amacıyla tasarlanmıştır. Koleksiyon, araç bazlı tekil kayıt yapısı ile çalışmakta ve vehicle_id alanı üzerinde **unique index** barındırmaktadır.

Başlıca alanlar:

- **vehicle_id**: Araç kimliği (ör. musoshi003)
- **remaining_energy**: Kalan enerji miktarı
- **nonlinear_energy**: Segment bazlı hesaplanan enerji tüketimi
- **segment_count**: Kat edilen yol segment sayısı
- **last_updated**: Son güncelleme zamanı

Bu koleksiyon, **Fleet Monitoring** modülünde gerçek zamanlı enerji takibi ve **Performance Analytics** modülünde enerji verimliliği analizleri için kullanılmaktadır.

23.1.3 Sipariş ve Operasyonel Veriler

RouteManagementDB içerisinde aşağıdaki koleksiyonlar, sipariş yaşam döngüsünü ve saha operasyonlarını desteklemektedir:

- **orders / Orders / Orders_S**: Siparişlerin farklı aşamalardaki temsilleri
- **customers**: Müşteri bilgileri
- **Products**: Taşınan ürünler
- **Tasks**: Siparişlere bağlı görev tanımları
- **OrderStatus**: Sipariş durum geçişleri
- **routes / route4vehicles**: Rota–araç eşleşmeleri
- **Depot**: Başlangıç ve bitiş noktaları
- **alerts**: Operasyon sırasında üretilen uyarılar
- **performances / anlikPerformansRota**: Anlık ve geçmiş performans ölçümleri

Bu yapı, **sipariş** → **rota** → **araç** → **enerji** → **performans** zincirinin MongoDB tarafındaki karşılığını oluşturmaktadır.

24. Rotalama Algoritmaları (Diğer)

Rota optimizasyon algoritmaları için çalışan servisler docker konteyner veya uvicorn server olarak başlatılmaktadır. Bu algoritmalardan SA-TS-ALNS algoritmaları dışındakiler single thread çalışırken bu algoritmaların olduğu servis multi-thread olarak çalışmaktadır.

24.1 Multi-Thread SA-TS-ALNS Servisi

Filo yönetim yazılımı için aynı anda birden çok isteğe cevap verebilmesi için bu servis asenkron ve multi-process mimariye uygun hale getirilmiştir. Bu yapılandırma, özellikle hesaplama yoğunluğu yüksek olan (CPU-bound) SA (Simulated Annealing), TS (Tabu Search) ve ALNS algoritmalarının, API'nin ana olay döngüsünü (event loop) bloklamadan çalışmasını sağlar.

Sistemin çalışma prensibi ve teknik detayları aşağıdadır:

1. Gerçek Paralellik (ProcessPoolExecutor): Python'un standart çalışma yapısında bulunan Global Interpreter Lock (GIL) sınırlamasını aşmak için servis, *concurrent.futures.ProcessPoolExecutor* kullanmaktadır.

- Uygulama ayağa kalkarken (*startup_event*), sistem kaynaklarına göre belirlenen sayıda (kodda: 2) "Worker Process" (İşçi Süreç) hazırlar.
- Gelen her optimizasyon isteği, ana sunucuyu meşgul etmeden bu boşta bekleyen işçilerden birine atanır.
- Bu sayede sunucu bir yandan ağır matematiksel hesaplamaları yaparken, diğer yandan yeni gelen HTTP isteklerine (Health check, yeni iş emri vb.) cevap vermeye devam edebilir.

2. İstek ve Veri İzolasyonu (Request Scoping): Paralel çalışan sistemlerde verilerin birbirine karışmaması kritik öneme sahiptir. Servis bu güvenliği şu adımlarla sağlar:

- **Unique ID:** Her gelen istek için benzersiz bir kimlik (UUID) oluşturulur.
- **Dinamik Dosya Yapısı:** *BASE_REQUEST_FOLDER* altında her UUID için özel klasörler (*Input*, *Dataset*, *SolutionFiles*) yaratılır.
- **İşlem:** Algoritma sadece kendine ayrılan bu izole alandaki dosyaları okur ve yazar. İşlem bittikten sonra sonuçlar ziplenir ve arka planda çalışan temizlik mekanizması (*cleanup_after_response*) ile geçici dosyalar sunucudan silinir.

3. Hata Toleransı ve İzlenebilirlik:

- **Startup Warm-up:** Servis başlarken worker'ları test eder (dummy task). Eğer worker'lar başlatılamazsa sistem otomatik olarak senkron (sıralı) moda düşerek hizmetin kesilmesini engeller (*Fallback to synchronous mode*).
- **Health & Status:** */health* ve */status* uç noktaları üzerinden o an kaç işçinin aktif olduğu ve sistemin yük durumu anlık olarak izlenebilir.

4. Desteklenen Algoritmalar: Servis tek bir çatı altında aşağıdaki algoritmaları paralel olarak çalıştırabilir:

- **ALNS** (Adaptive Large Neighborhood Search)
- **SA** (Simulated Annealing)
- **TS** (Tabu Search)

24.2 Heterojen Filolu Rotalama için Algoritma

Bu servis, farklı özelliklere (örneğin; farklı kapasite, yakıt tipi, menzil vb.) sahip araçlardan oluşan filolar için **optimize edilmiş dağıtım rotaları** hesaplar.

Özellikle **Heterojen Filo (Karma Filo)** yönetimi için geliştirilmiştir; yani filonuzda hem kamyonet hem elektrikli hem benzinli araçlar varsa, bu algoritma her aracın kapasitesine uygun görev ataması yapar. Algoritma aşağıdaki servis bilgilerine sahiptir.

Parametre	Değer
Yöntem	`POST`
Endpoint	`/start_alns`
Port	`8013` (Varsayılan)
Veri Formatı	RoutingML

Benzer girdiler ile çalışan algoritma RoutingML v1.05 versiyonu ile entegredir. Burada araç bilgilerinin daha detaylı şekilde algoritmaya verilmesi ile araca göre atamalar yapılır. Bu algoritma için araç bilgileri, algoritma detayları ve diğer gereksinimler hakkında detaylı bilgi için Fleet Management Revision Requirements (16.10.2025) - v3.docx dökümanını inceleyebilirsiniz.

24.3 Dinamik Rotalama için Algoritma

Gerçek bir ortamda teslimata başlandığında da yeni siparişler gelmeye devam edebilir. Bu durumda bu siparişlerin sonraki plana bırakılması müşteri memnuniyeti kadar maliyet açısından da olumsuz etkiler yaratabilir. Bu nedenle teslimatına devam etmekte olan araçlara yeni siparişlerin atanması ve rotalarının güncellenmesi oldukça önemli olmaktadır. Bu amaçla oluşturulan dinamik rotalama algoritması servisi, **gerçek zamanlı (dinamik)** teslimat ihtiyaçları için geliştirilmiştir. Standart rotalama algoritmalarından farklı olarak, **mevcut aktif rotalara** sonradan gelen **yeni siparişleri** entegre eder.

Özellikle **"Last Mile Delivery"** ve **Anlık Kurye** hizmetleri gibi operasyonun sürekli değiştiği senaryolar için idealdir.

Parametre	Değer
Yöntem	`POST`
Endpoint	`/start_dynamic_routing`
Port	`8011` (Varsayılan)
Veri Formatı	RoutingML + Önceki çözümün Route4Vehicle dosyası

Burada algoritma girdileri RoutingML v1.05 için uygun olarak diğer algoritmalara benzerdir. Farklı olan kısmı siparişlerin tamamı yerine dinamik siparişin özel formatı olan "Info4Task" yapısında girdi alır. Ayrıca araçların mevcut rotalarını ve hangi teslimatların tamamlandığını içeren Route4Vehicle dosyasını da alır. Böylece hangi rotanın/aracın müsait olduğuna göre atama yapılarak çözüm üretilir. Çıktı tarafında gönderilen yapı diğer algoritmalar ile aynıdır.

24.4 Rotalama Algoritmalarının Girdi ve Çıktı Standartı – Routing Markup Language (RoutingML)

Filo yönetimi ve rotalama optimizasyonu problemlerinde kullanılan algoritmalar; araç özellikleri, müşteri talepleri, çevresel koşullar ve optimizasyon parametreleri gibi çok sayıda heterojen girdiye ihtiyaç duymaktadır. Literatürde ve endüstriyel uygulamalarda bu girdiler genellikle algoritmaya veya yazılım mimarisine özel veri yapıları ile temsil edilmekte, bu durum farklı algoritmaların karşılaştırılmasını, entegrasyonunu ve yeniden kullanılabilirliğini zorlaştırmaktadır.

Bu problemi adreslemek amacıyla **Routing Markup Language (RoutingML)** geliştirilmiştir. RoutingML, rotalama troubled sorunları için **algoritmadan bağımsız**, **genişletilebilir** ve **standartlaştırılmış** bir girdi/çıktı tanımlama dili sunar.

24.4.1 RoutingML'in Amacı ve Tasarım İlkeleri

RoutingML aşağıdaki temel hedefler doğrultusunda tasarlanmıştır:

- **Algoritma bağımsızlığı:** Aynı problem tanımı, farklı çözüm yaklaşımları (exact, meta-sezgisel, öğrenme tabanlı) ile kullanılabilir.
- **Modülerlik:** Araç, çevre, algoritma ve görev tanımları birbirinden ayrık ve yeniden kullanılabilir yapıdadır.
- **Genişletilebilirlik:** Yeni araç tipleri, enerji modelleri veya algoritma parametreleri şema bozulmadan eklenebilir.
- **Doğrulanabilirlik:** XML Schema (XSD) desteği ile veri bütünlüğü ve format doğruluğu garanti altına alınır.
- **Sistemler arası entegrasyon:** Optimizasyon motorları, simülasyon ortamları ve filo yönetim sistemleri arasında ortak bir arayüz sağlar.

24.4.2 RoutingML Genel Yapısı

RoutingML, XML tabanlı bir işaretleme dili olarak yapılandırılmıştır ve temel olarak dört ana bileşenden oluşur:

1. **Problem Tanımı (Problem Definition)**
2. **Araç Tanımı (Vehicle Definition)**
3. **Algoritma Tanımı (Algorithm Definition)**
4. **Çözüm / Çıktı Tanımı (Solution Definition)**

Bu yapı sayesinde bir rotalama problemi, girdi ve çıktı düzeyinde açık ve izlenebilir bir şekilde ifade edilebilir.

24.4.3 Girdi Yapısı (Input Specification)

RoutingML girdi dokümanı, optimizasyon algoritmasının ihtiyaç duyduğu tüm bilgileri kapsar.

a) Görev ve Müşteri Tanımları

- Talep miktarı
- Zaman pencereleri
- Hizmet süresi

- Konum bilgileri (koordinat, düğüm ID)

b) Araç Tanımları

- Araç tipi (EV, ICE, hibrit)
- Kapasite bilgileri
- Enerji/batarya özellikleri
- Tüketim modelleri
- Şarj ve operasyonel kısıtlar

c) Çevresel ve Ağ Bilgileri

- Yol ağı
- Mesafe ve süre matrisleri
- Trafik veya hız profilleri
- Şarj istasyonu bilgileri (konum, kapasite, güç)

d) Algoritma Konfigürasyonu

- Algoritma türü (ör. exact, ALNS, RL)
- Amaç fonksiyonları
- Kısıt setleri
- Parametreler ve hiperparametreler

Bu yapı, aynı problem tanımının farklı algoritmalarla **hiçbir veri dönüşümü gerektirmeden** çalıştırılmasına olanak tanır.

24.4.4 Çıktı Yapısı (Output Specification)

RoutingML çıktıları, algoritma sonucunda üretilen çözümün sistematik ve standart bir biçimde temsil edilmesini sağlar.

Çıktı dokümanı aşağıdaki bilgileri içerir:

- Araç bazlı rota sıralamaları
- Ziyaret edilen düğümler ve zaman çizelgesi
- Enerji durumu (SoC) değişimleri
- Şarj durakları ve süreleri
- Toplam maliyet, süre ve enerji metrikleri
- Çözümün fizibilite durumu

Bu standart yapı, çözümlerin:

- Görselleştirilmesi,
- Simülasyon ortamlarında test edilmesi,
- Farklı algoritmalar arasında karşılaştırılması gibi işlemleri kolaylaştırır.

Teknik detaylar ve şema yapıları ile birlikte örnek XML yapılarını görmek için RoutingXML Document_v1.05_15102025.docx dökümanını inceleyebilirsiniz.